GENOME FORGE TUTORIAL

Teach Yourself Bioinformatics with Genome Forge

Textbook edition · self-study with real molecular data

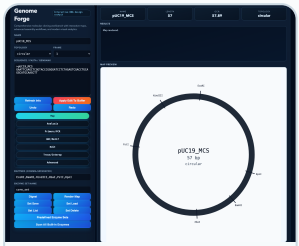
GENOME FORGE V0.1.8 · TEXTBOOK EDITION

Teach Yourself Bioinformatics with Genome Forge

A self-study bioinformatics text for engineers, analysts, and scientists who want to learn from real laboratory molecules rather than toy examples.

The course uses public-source records such as EGFP, mCherry, pUC19/lacZ logic, and a BRAF exon 15 hotspot fragment. Clearly labelled training derivatives appear only when they sharpen a teaching goal, such as variant interpretation, family-wide assay design, or ambiguity-aware search.

Mode	Cases	Audience	Release
Self-study course	**39 total lessons**	**CS to biology bridge**	**2026-04-27**

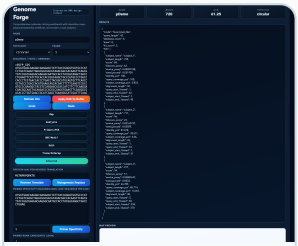


CASE A

Restriction-map workflow in the live UI

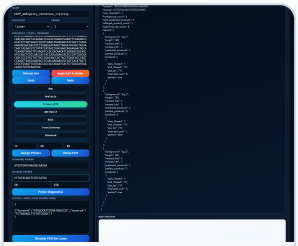
CASE AH

Chromatogram-first review workflow



CASE AJ

BLAST-like identity search workflow



CASE AL

Degenerate-primer assay workflow

Each lesson combines procedure, expected results, biological interpretation, and a clear statement of why the data matter in practice.

IMPRINT

Edition and Copyright

Title: *Teach Yourself Bioinformatics with Genome Forge*

Edition: Genome Forge v0.1.8 textbook edition generated on `2026-04-27`.

Authoring body: Genome Forge contributors

Repository: https://github.com/felizvida/genomeforge

License: Apache License 2.0 for the project source. Public-source records and clearly labelled training derivatives are documented in the bundled dataset metadata.

Scope: This volume contains 39 lessons, real-world sample data, and HTML/PDF outputs rebuilt from the same source tutorial.

Suggested citation: Genome Forge contributors. *Teach Yourself Bioinformatics with Genome Forge*. Genome Forge v0.1.8. 2026.

FRONT MATTER

Publication Notes

Abstract

This volume teaches practical bioinformatics with Genome Forge through real biological records, stepwise software workflows, expected outputs, interpretation guidance, and biological explanation in one reproducible text.

The current edition contains 39 lessons organized into clusters that move from molecular architecture and restriction logic to assay design, assembly, comparative reasoning, ambiguity-aware analysis, and reproducible project delivery.

Edition and Citation

Edition: Genome Forge Textbook Edition, generated from repository source on `2026-04-27`.

Preferred citation: *Teach Yourself Bioinformatics with Genome Forge*, Genome Forge v0.1.8, tutorial edition.

Formats: HTML and PDF are generated from the same source so case numbering, sample data, and screenshots stay aligned.

COURSE MAP

Learning Path

The tutorial is organized as a reasoning sequence: identify the biological object, design the experiment, evaluate the evidence, then package the work so another person can trust it.

COURSE ARC

39 lessons, one practical reasoning loop

Each case is designed to move from software action to biological claim. The point is not button memorization; it is learning how sequence evidence changes an experimental decision.

I **Read the molecule**

Map, annotate, translate, and decide what kind of biological object is in front of you.

II **Design the experiment**

Choose primers, enzymes, assemblies, and edits with the failure modes visible.

III **Interrogate evidence**

Use traces, alignments, search, coverage, and ambiguity codes to decide what the data support.

IV **Package the work**

Preserve projects, reviews, share bundles, and explanations so someone else can continue cleanly.

USING THIS EDITION

How to Use This Edition

Quickstart

1. Start the web UI with `python3 web_ui.py --port 8080`.

2. Materialize a case bundle with `python3 docs/tutorial/datasets/extract_case_bundle.py --case A --out ./tmp/genomeforge_case_a`.

3. Open `http://127.0.0.1:8080`, load the FASTA from your case bundle, and follow the matching case steps below.

What This Edition Adds

This edition explains what the input data are, why the task matters biologically, what a meaningful result looks like, and what you should not conclude from the output.

It is built so the numbers point to a real scientific story, and the ambiguity-aware methods are taught directly rather than left implicit.

ORIENTATION

Meaningful Results Preview

These quick facts are derived directly from the bundled records and serve as a calibration point before you begin the 39 lessons.

EGFP is a clean coding-sequence teaching record

720 bp → 239 aa + stop

That makes it ideal for learning frame-aware translation, variant annotation, and plasmid verification without the extra ambiguity of introns or splice context.

The pUC19 multiple-cloning site is densely engineered

57 bp with 6 common unique sites

This tiny region packs a surprising amount of experimental flexibility into a few dozen bases, which is why it became a cloning-era classic.

A one-codon EGFP derivative can still be biologically dramatic

EGFP vs Y67H-like variant: 99.861% nucleotide identity

The tutorial uses this to teach a core lesson in molecular biology: a small sequence delta can carry a large phenotype when it lands in a privileged site.

The BRAF training fragment is genomic context, not a standalone CDS

196 bp with 2 naive frame-1 stop codons

That is exactly what makes it useful. It forces you to distinguish "this DNA is wrong" from "this DNA is a different biological object than a clean coding sequence."

Reporter proteins can do similar jobs while having different sequence histories

EGFP length 720 bp vs mCherry length 711 bp

Comparing them is a good reminder that "same use in the lab" does not imply "same sequence architecture" or even the same engineering tradeoffs.

Genome Forge now teaches uncertainty as a first-class sequence state

EGFP ambiguity training record carries 3 explicit unresolved positions

That matters because real assay design and identity search often start before every position is perfectly resolved. Good workflows preserve uncertainty instead of flattening it away.

DATA

How to Use the Sample Data

Bundled Data Files

- `docs/tutorial/datasets/training_real_world_sequences.fasta`: base public-source sequences.
- `docs/tutorial/datasets/training_real_world_dataset.json`: metadata, sources, case inputs, and derived-record definitions.
- `docs/tutorial/datasets/case_playbook.md`: compact case-by-case checklist.
- `docs/tutorial/datasets/case_bundles/`: prebuilt ready-to-load bundles for all 39 tutorial cases.
- `docs/tutorial/datasets/extract_case_bundle.py`: writes ready-to-run per-case FASTA bundles.

One Good Workflow Habit

Always save the exact case bundle you used. That keeps the tutorial reproducible and avoids "I think I loaded the right sequence" problems. Every case already ships with a prebuilt bundle, so you can start quickly and still regenerate it later if you want to inspect provenance.

```
python3 docs/tutorial/datasets/extract_case_bundle.py --case K --out ./tmp/genomeforge_case_k
```

STUDY METHOD

How to Study This Book

Read the data type first

Before you run anything, identify whether the input is a coding sequence, genomic fragment, plasmid-like construct, chromatogram-derived consensus, or uncertainty-bearing record. Most downstream mistakes come from treating those as interchangeable.

Use the sample results as calibration, not as a cheat sheet

The sample results tell you what a believable answer should feel like. They do not replace your own run. A good habit is to compare your output to the sample and ask why any difference exists.

Write down the biological claim separately from the software output

The result is not just “the tool said X.” The result is the biological sentence you can defend after seeing X. That distinction is what turns software use into bioinformatics reasoning.

REFERENCE

Primer on Ambiguity Codes

Several later lessons teach ambiguity-aware matching directly. These symbols do not mean the sequence is broken. They mean the evidence still permits a small set of bases at a position, and Genome Forge can search, compare, and design around that uncertainty.

Code	Allowed base(s)	Meaning	Why you would keep it
`A`	A	Adenine only	Exact called base or exact assay requirement.
`C`	C	Cytosine only	Exact called base or exact assay requirement.
`G`	G	Guanine only	Exact called base or exact assay requirement.
`T`	T	Thymine only	Exact called base or exact assay requirement.
`R`	A or G	Purine	Useful when a site varies between adenine and guanine.
`Y`	C or T	Pyrimidine	Common in mixed trace calls and degenerate primers.
`S`	G or C	Strong pair	Both options make three hydrogen bonds to the complement.
`W`	A or T	Weak pair	Both options make two hydrogen bonds to the complement.
`K`	G or T	Keto	Used when the target family tolerates a purine/pyrimidine swap at one site.
`M`	A or C	Amino	Useful for family-wide assay design.
`B`	C or G or T	Not A	Represents uncertainty while still excluding one base.
`D`	A or G or T	Not C	Represents uncertainty while still excluding one base.
`H`	A or C or T	Not G	Represents uncertainty while still excluding one base.
`V`	A or C or G	Not T	Represents uncertainty while still excluding one base.

Code	Allowed base(s)	Meaning	Why you would keep it
N	A or C or G or T	Any base	Used when the evidence does not justify a more specific call.

Why ambiguity is honest

Forcing an uncertain position to one exact base may look cleaner, but it destroys evidence. Ambiguity codes preserve what the data still allow.

Why assay design cares

Degenerate primers use these symbols on purpose so one assay can still cover a small family of related templates.

Why search still works

An uncertainty-bearing query can still identify the correct molecule family if the unresolved positions are represented explicitly instead of hidden.

INTERFACE

Visual Tour of the Workbench

These illustrations help you recognize what Genome Forge is showing in each workflow: structure, evidence, divergence, and provenance.

FIGURE 1.

Restriction-map reasoning

A plasmid map is only useful when it helps you choose a safe cloning move. This view is strongest when paired with feature context and digest logic.

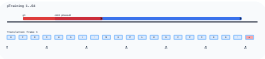


FIGURE 2.

Sequence-track thinking

The whole point of a sequence-track view is to keep letters, codons, amino acids, and annotations in one frame of reference.

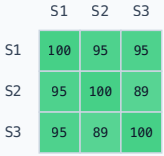


FIGURE 3.

Family-scale comparison

Multiple alignment becomes biologically meaningful when you can point to a divergence hotspot and explain what the changed site may do.

FIGURE 4.

Provenance and history

Sequence work gains credibility when the molecular state and the decision trail stay attached to each other.

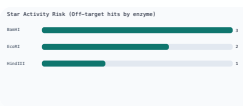


FIGURE 5.

Assay risk modeling

Good assay design is not just finding a candidate that works once. It is anticipating how the workflow can fail.

FIGURE 6.

Assembly visualization

Assembly views help translate overlap logic into a believable final construct rather than a hand-wavy plan.

BIOLOGICAL OBJECTS

Real-World Record Field Guide

These are the biological objects that power the tutorial. Some are public-source sequences bundled directly in the FASTA file. Others are clearly labelled training derivatives created so specific comparison cases have an answer key.

Record	Origin	Why it matters	Input data explained	Source
`EGFP_CDS` public-source	Engineered fluorescent reporter derived from the Aequorea victoria GFP family.	EGFP is a canonical lab reporter: great for teaching translation, cloning, sequence verification, and how a few codons can change an optical phenotype.	This is a protein-coding DNA sequence (CDS). The sequence is meant to be translated from the first base, so codon boundaries and reading frame matter immediately.	PubMed: enhanced GFP mutants overview
`mCherry_CDS` public-source	Monomeric red fluorescent protein from the mFruit engineering lineage.	mCherry is a real-world counter-example to GFP: same broad job as a reporter, different sequence history, color, and engineering constraints.	This is also a CDS, but it encodes a coral-derived red fluorescent protein rather than a GFP-family green reporter. That makes it useful for pairwise comparison and identity search.	PubMed: A monomeric red fluorescent protein
`pUC19_MCS` public-source	The pUC19 multiple-cloning site from one of the most widely used teaching and cloning vectors in molecular biology.	A dense multiple-cloning site is a perfect example of engineered sequence architecture: every base exists to make cloning more flexible.	This sequence is short, circularly interpreted, and packed with restriction motifs. It is intentionally synthetic and engineered for manipulation rather than natural gene expression.	NCBI Nucleotide: pUC19 complete sequence
`lacZ_alpha_fragment` public-source	lacZ alpha fragment from the classic blue-white screening system associated with cloning vectors such as pUC19.	This fragment is a perfect teaching example of phenotype-linked DNA: inserting the wrong thing into the wrong place changes colony color.	The sequence is a cloning-era workhorse. It contains coding material, but many workflows care more about its role as a reporter module than as a standalone protein-coding fragment.	NCBI Nucleotide: pUC19 complete sequence

Record	Origin	Why it matters	Input data explained	Source
`BRAF_exon15_fragment` public-source	A hotspot-rich fragment centered on human BRAF exon 15, the region associated with the famous V600 oncogenic mutation family.	This is the most medically consequential sequence in the training set. It is ideal for primer design, genotyping logic, CRISPR planning, and explaining why genomic DNA is not the same thing as a CDS.	This is a genomic fragment, not a clean coding-sequence input. If you translate it naively, you hit stop codons because intron/exon context and strand assumptions matter.	NCBI Gene: BRAF (human)
`EGFP_Y67H_training_variant` derived-training	Training derivative of EGFP that changes the aromatic residue in the chromophore-forming motif.	This is not a random mutation. It models the idea that one codon change near a chromophore can produce a large optical shift, which is a powerful lesson for variant interpretation.	Because this record is derived from EGFP by a single codon change, it is excellent for pairwise alignment, amino-acid consequence analysis, and demonstrating how "small diff, big phenotype" problems happen in biology.	Derived from EGFP mutant logic described in GFP engineering literature
`EGFP_S204Y_training_variant` derived-training	Training derivative of EGFP that alters an aromatic-packing site near the chromophore environment.	It gives the tutorial a second closely related variant, which makes alignment, consensus, and comparison-lens examples far more realistic than comparing unrelated proteins only.	Like the Y67H-like variant, this record is generated from a public-source EGFP backbone. The point is not to claim a specific commercial reagent but to give you a realistic engineering-style mutation to reason about.	Derived from EGFP engineering principles
`EGFP_ambiguity_consensus_training` derived-training	Training derivative of EGFP that uses IUPAC ambiguity symbols to mimic a consensus sequence assembled from uncertain or mixed evidence.	It teaches that uncertainty can be represented explicitly instead of being hidden behind a forced single-base call.	This record is still an EGFP-like coding sequence, but a few positions are encoded as ambiguity symbols such as R, Y, and N. That means the sequence stands for a small set of plausible molecules rather than one exact DNA string.	Derived from EGFP_CDS for ambiguity-aware assay and search training

Reporter Biology

EGFP and mCherry let you practice coding-sequence analysis on records that many labs actually clone, image, and verify.

Cloning Architecture

pUC19 MCS and lacZ alpha turn restriction logic into an experimentally meaningful story because the vector design is tied to blue-white screening.

Disease-Linked DNA

The BRAF fragment keeps the course grounded in medically important sequence interpretation, not only reporter-gene demos.

CONTENTS

Table of Contents

Recommended order for readers new to biology: Cluster A → B → C → D → G → E → F → H.

INTERPRETATION

How to Read a Bioinformatics Result Like a Scientist

Start with the biological question

Ask what decision the output is supposed to support. A beautiful visualization is not useful if it does not change a real experimental choice.

Respect the input data type

A genomic fragment, a coding sequence, a plasmid map, and a chromatogram are not interchangeable. The same algorithm can be correct and still be answering the wrong question.

Separate observation from inference

Report what the tool measured first, then explain what you think that measurement means biologically, and finally say how confident you are.

CLUSTER A

Cluster A: Molecule Architecture and Restriction Logic

How a DNA molecule is physically organized and where you can safely cut or modify it.

Lessons in This Cluster

Case A · Restriction Map for Cloning Entry Design

Case B · Methylation-Aware Digest Interpretation

Case C · Star Activity Risk Review

Case U · k-mer Profile for Contamination Suspicion

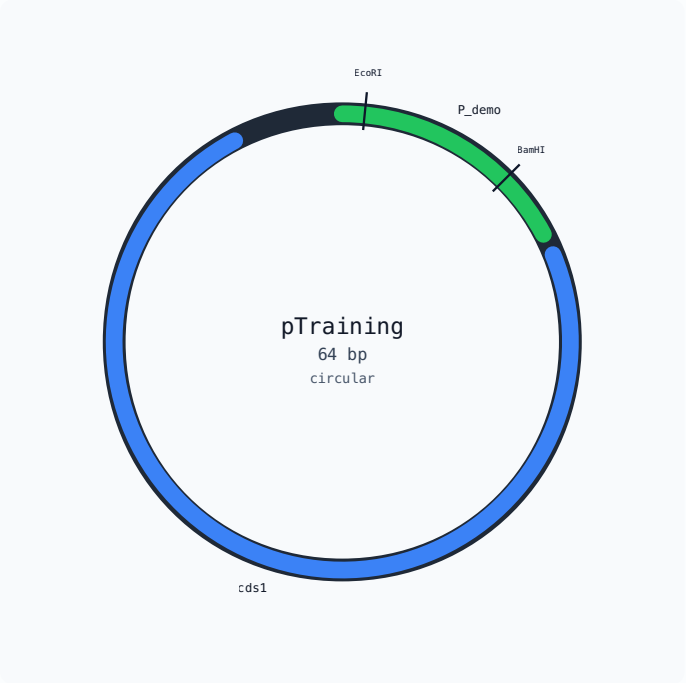


Figure 7. Circular and linear sequence views are most useful when you interpret them as future wet-lab decisions, not just graphics.

CASE A · CLUSTER A

Case A: Restriction Map for Cloning Entry Design

Which enzyme pair opens the vector cleanly without compromising the blue-white screening logic built around lacZ alpha?

TAB

Map

WORKFLOW

Render the pUC19 map and compare unique restriction choices before adding a reporter insert.

RECORDS

- pUC19_MCS
- lacZ_alpha_fragment

APIS

`/api/map` `/api/digest`

Why This Case Matters

You are loading a real vector architecture rather than a random string. The pUC19 multiple-cloning site is deliberately dense with restriction motifs, and the adjacent lacZ alpha fragment is what gives the classic blue/white colony readout. Those two pieces together teach why a plasmid map is really an experimental design document.

Restriction mapping matters because plasmid biology is modular. The MCS exists to be cut, but the neighboring reporter logic exists to be preserved or intentionally disrupted. A good cloning map therefore answers two questions at once: where can I cut, and what biological readout will survive afterward?

Input Data Explained

This sequence is short, circularly interpreted, and packed with restriction motifs. It is intentionally synthetic and engineered for manipulation rather than natural gene expression. The sequence is a cloning-era workhorse. It contains coding material, but many workflows care more about its role as a reporter module than as a standalone protein-coding fragment.

The pUC19 multiple-cloning site is only a few dozen bases long, but it is one of the most recognizable pieces of engineered DNA in molecular biology.

Step-by-Step in Genome Forge

1. Use the included prebuilt bundle `docs/tutorial/datasets/case_bundles/case_a/records.fasta` or regenerate it with `python3 docs/tutorial/datasets/extract_case_bundle.py --case A --out ./tmp/genomeforge_case_a`.
2. Open the `Map` tab in Genome Forge and load: `pUC19_MCS, lacZ_alpha_fragment`.
3. Run `Render the pUC19 map and compare unique restriction choices before adding a reporter insert.` once with default settings, then rerun after changing the enzyme panel.
4. Capture one screenshot of the main result panel so you can compare your run with the sample interpretation later.
5. Record the relevant endpoint(s): `/api/map, /api/digest` and write one sentence explaining the biological takeaway.

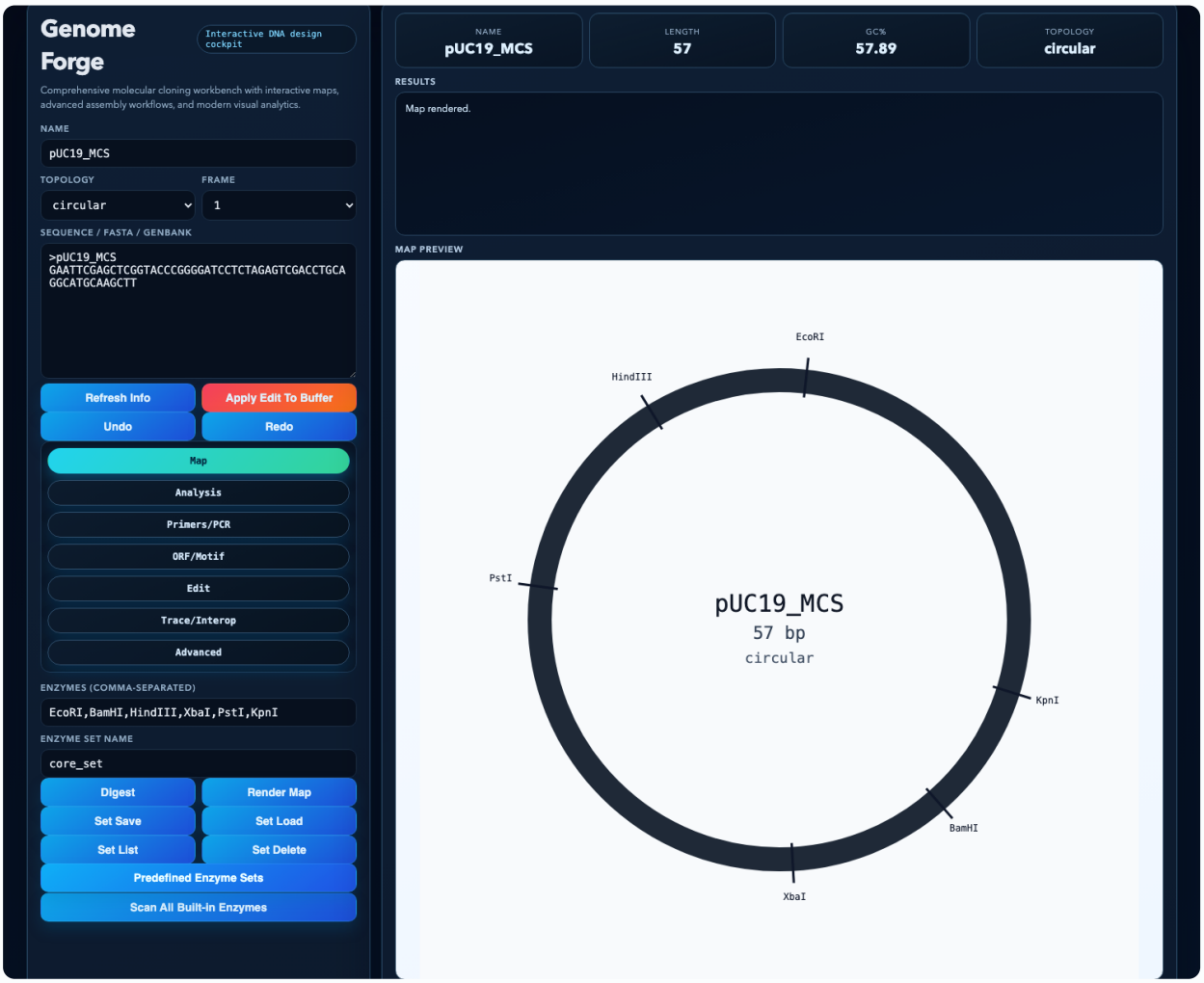


Figure 8. Restriction-map workflow in the live UI. Real UI screenshot from the bundled pUC19 MCS case. The map is rendered with a common cloning enzyme panel so you can see which sites are unique before choosing a directional cut strategy.

Sample Results

Representative output shaped around the bundled real-world record(s) or their documented training derivatives. Values are rounded for readability, but the biological story is tied to the included data.


```
{
  "unique_sites": [
    "EcoRI",
    "BamHI",
    "HindIII",
    "XbaI",
    "PstI",
    "KpnI"
  ],
  "best_directional_pair": [
    "EcoRI",
    "BamHI"
  ],
  "lacZ_alpha_screen_preserved_until_insert": true
}
```

Expected Results

- A circular map with the multiple-cloning site and reporter context clearly marked.
- A shortlist of unique cut sites or directional cut pairs that can linearize the vector safely.
- A written decision explaining which enzyme choice best supports the intended cloning strategy.

How to Interpret the Results

- Unique cut sites are valuable because they open the vector once and only once.
- A directional pair is stronger than two random unique sites because it helps enforce insert orientation.
- If your chosen sites overlap a feature you depend on, the map is warning you before the bench can do it expensively.

Biological Explanation

Restriction mapping matters because plasmid biology is modular. The MCS exists to be cut, but the neighboring reporter logic exists to be preserved or intentionally disrupted. A good cloning map therefore answers two questions at once: where can I cut, and what biological readout will survive afterward?

Fun fact from this example:

The pUC19 multiple-cloning site is only a few dozen bases long, but it is one of the most recognizable pieces of engineered DNA in molecular biology.

CASE B · CLUSTER A

Case B: Methylation-Aware Digest Interpretation

Can a sequence that looks correct on paper still digest differently because the DNA was prepared in a methylating host?

TAB

Advanced

WORKFLOW

Compare standard digest output with methylation-aware digest logic on a real cloning vector motif set.

RECORDS

`pUC19_MCS`

APIS

`/api/digest-advanced`

Why This Case Matters

The same pUC19-derived motif map is now interpreted with chemistry layered on top. The important input is not only the recognition site but also whether that site can be chemically masked by methylation, which is common in routine plasmid prep workflows.

This case is one of the best ways to teach a computer scientist that biology has state beyond the ASCII sequence. The DNA letters are unchanged, yet the molecular behavior changes because an enzyme cannot access the site it expects.

Input Data Explained

This sequence is short, circularly interpreted, and packed with restriction motifs. It is intentionally synthetic and engineered for manipulation rather than natural gene expression.

Many “mysterious digest failures” are actually host-state stories, not sequence-identity stories.

Step-by-Step in Genome Forge

1. Use the included prebuilt bundle `docs/tutorial/datasets/case_bundles/case_b/records.fasta` or regenerate it with `python3 docs/tutorial/datasets/extract_case_bundle.py --case B --out ./tmp/genomeforge_case_b`.
2. Open the `Advanced` tab in Genome Forge and load: `pUC19_MCS`.
3. Run `Compare standard digest output with methylation-aware digest logic on a real cloning vector motif set.` once with default settings, then rerun after changing the methylated motif list.
4. Capture one screenshot of the main result panel so you can compare your run with the sample interpretation later.
5. Record the relevant endpoint(s): `/api/digest-advanced` and write one sentence explaining the biological takeaway.

Sample Results

Representative output shaped around the bundled real-world record(s) or their documented training derivatives. Values are rounded for readability, but the biological story is tied to the included data.


```
{
  "blocked_cuts": [
    {
      "enzyme": "EcoRI",
      "motif": "GAATTC"
    }
  ],
  "remaining_visible_cuts": [
    "BamHI",
    "HindIII"
  ],
  "status": "revise host-or-enzyme plan"
}
```


Expected Results

- A digest report that lists both successful and blocked cut events.
- A fragment-length interpretation that changes after methylation is modeled.
- A troubleshooting conclusion that explains whether the digest discrepancy is chemically plausible.

How to Interpret the Results

- If a predicted cut disappears only in the methylation-aware run, the enzyme is not suddenly “wrong”; the substrate context changed.
- Blocked cuts near the cloning site can explain why a plasmid looks undigested or partially digested on a gel.
- The practical fix is usually a different enzyme, a different host, or a different validation strategy.

Biological Explanation

This case is one of the best ways to teach a computer scientist that biology has state beyond the ASCII sequence. The DNA letters are unchanged, yet the molecular behavior changes because an enzyme cannot access the site it expects.

Fun fact from this example:

Many “mysterious digest failures” are actually host-state stories, not sequence-identity stories.

CASE C · CLUSTER A

Case C: Star Activity Risk Review

If reaction conditions get sloppy, where would near-miss cuts land and which of those cuts would actually hurt the experiment?

TAB

Advanced

WORKFLOW

Scan relaxed-matching cut risk to understand how star activity can create off-target restriction events.

RECORDS

- pUC19_MCS
- lacZ_alpha_fragment

APIS

- `/api/star-activity-scan`

Why This Case Matters

The inputs are the same real cloning elements as Case A, but now the sequence is being treated under a relaxed specificity model. That lets you connect enzyme biochemistry to the spatial layout of engineered vector features.

Star activity is a nice teaching bridge between exact pattern matching and biology under stress. Under non-ideal conditions, enzymes may behave like approximate matchers. The biologically important question is not just how many off-targets appear, but whether any land in irreplaceable regions.

Input Data Explained

This sequence is short, circularly interpreted, and packed with restriction motifs. It is intentionally synthetic and engineered for manipulation rather than natural gene expression. The sequence is a cloning-era workhorse. It contains coding material, but many workflows care more about its role as a reporter module than as a standalone protein-coding fragment.

A single risky off-target cut inside a key feature matters more than a long list of harmless near-matches elsewhere.

Step-by-Step in Genome Forge

1. Use the included prebuilt bundle `docs/tutorial/datasets/case_bundles/case_c/records.fasta` or regenerate it with `python3 docs/tutorial/datasets/extract_case_bundle.py --case C --out ./tmp/genomeforge_case_c`.
2. Open the `Advanced` tab in Genome Forge and load: `pUC19_MCS, lacZ_alpha_fragment`.
3. Run `Scan relaxed-matching cut risk to understand how star activity can create off-target restriction events.` once with default settings, then rerun after changing the mismatch tolerance.
4. Capture one screenshot of the main result panel so you can compare your run with the sample interpretation later.
5. Record the relevant endpoint(s): `/api/star-activity-scan` and write one sentence explaining the biological takeaway.

Sample Results

Representative output shaped around the bundled real-world record(s) or their documented training derivatives. Values are rounded for readability, but the biological story is tied to the included data.


```
{
  "star_hit_count": 4,
  "highest_risk_region": "lacZ_alpha boundary",
  "recommended_response": "tighten reaction conditions or switch enzyme"
}
```


Expected Results

- A ranked list of possible star-activity sites and their mismatch burden.
- A spatial interpretation of whether any near-miss cut touches essential cloning features.
- A conservative recommendation for reducing digest risk.

How to Interpret the Results

- A low total count is not automatically safe if one site lands in a fragile region.
- Star activity is a risk model, not proof that the event happened; treat it as an opportunity to design out uncertainty.
- The strongest conclusion combines off-target count, target location, and how much the experimental readout depends on that region.

Biological Explanation

Star activity is a nice teaching bridge between exact pattern matching and biology under stress. Under non-ideal conditions, enzymes may behave like approximate matchers. The biologically important question is not just how many off-targets appear, but whether any land in irreplaceable regions.

Fun fact from this example:

A single risky off-target cut inside a key feature matters more than a long list of harmless near-matches elsewhere.

CASE U · CLUSTER A

Case U: k-mer Profile for Contamination Suspicion

Does the sequence composition look like one coherent construct, or does it hint that two familiar lab molecules were mixed together?

TAB

Search

WORKFLOW

Use motif/entity search patterns to ask whether a supposed single-template sample smells like a mixed cloning population.

RECORDS

- EGFP_CDS
- mCherry_CDS
- pUC19_MCS

APIS

`/api/motif` `/api/search-entities`

Why This Case Matters

This case deliberately mixes a reporter-centric mental model with a vector-centric one. EGFP, mCherry, and pUC19 are all real lab molecules that often coexist on the same bench, which makes them realistic contamination suspects.

Contamination is often easier to suspect at the pattern level than at the base-call level. If a sample contains motifs or feature signatures from mutually incompatible molecules, the safest interpretation is that the sample identity problem comes before any downstream design work.

Input Data Explained

This is a protein-coding DNA sequence (CDS). The sequence is meant to be translated from the first base, so codon boundaries and reading frame matter immediately. This is also a CDS, but it encodes a coral-derived red fluorescent protein rather than a GFP-family green reporter. That makes it useful for pairwise comparison and identity search. This sequence is short, circularly interpreted, and packed with restriction motifs. It is intentionally synthetic and engineered for manipulation rather than natural gene expression.

The most useful contamination clue is often not a mismatch; it is a motif or feature that simply should not coexist with the biology you thought you had.

Step-by-Step in Genome Forge

1. Use the included prebuilt bundle `docs/tutorial/datasets/case_bundles/case_u/records.fasta` or regenerate it with `python3 docs/tutorial/datasets/extract_case_bundle.py --case U --out ./tmp/genomeforge_case_u`.
2. Open the `Search` tab in Genome Forge and load: `EGFP_CDS, mCherry_CDS, pUC19_MCS`.
3. Run `Use motif/entity search patterns to ask whether a supposed single-template sample smells like a mixed cloning population.` once with default settings, then rerun after changing the motif query or background panel.
4. Capture one screenshot of the main result panel so you can compare your run with the sample interpretation later.
5. Record the relevant endpoint(s): `/api/motif, /api/search-entities` and write one sentence explaining the biological takeaway.

Sample Results

Representative output shaped around the bundled real-world record(s) or their documented training derivatives. Values are rounded for readability, but the biological story is tied to the included data.


```
{
  "unexpected_feature_hits": [
    "mCherry-like motif",
    "pUC19-like restriction cluster"
  ],
  "contamination_hypothesis": "mixed template or carryover plasmid",
  "status": "quarantine sample"
}
```


Expected Results

- A motif/entity hit list that can be compared with the expected construct identity.
- A short contamination hypothesis grounded in actual known molecules from the training panel.
- A clear decision about whether to proceed or re-isolate the template.

How to Interpret the Results

- If the feature hits are coherent with one construct, proceed to finer analysis.
- If the hit pattern combines signatures from incompatible molecules, do not over-interpret downstream outputs.
- The cost of re-isolating a sample is usually much lower than the cost of trusting contaminated data.

Biological Explanation

Contamination is often easier to suspect at the pattern level than at the base-call level. If a sample contains motifs or feature signatures from mutually incompatible molecules, the safest interpretation is that the sample identity problem comes before any downstream design work.

Fun fact from this example:

The most useful contamination clue is often not a mismatch; it is a motif or feature that simply should not coexist with the biology you thought you had.

CLUSTER B

Cluster B: Sequence Meaning and Functional Annotation

How raw letters become genes, codons, proteins, and functional hypotheses.

Lessons in This Cluster

- Case D · Sequence Track and Translation Context
- Case M · ORF Scan and Coding Potential Triage
- Case P · Variant Annotation from Reference-Aligned Edits
- Case W · Protein Property Inference from Translation

Figure 9. Track views connect raw sequence, strand, feature, and codon logic in one place.

CASE D · CLUSTER B

Case D: Sequence Track and Translation Context

When you zoom in on a coding sequence, what exactly makes one nucleotide substitution harmless and another catastrophic?

TAB

Map

WORKFLOW

Inspect EGFP with sequence tracks so base coordinates, codons, amino acids, and features can be read together.

RECORDS

`EGFP_CDS`

APIS

`/api/sequence-tracks`

Why This Case Matters

EGFP is ideal here because it is a clean CDS with a familiar biological output. Every base belongs to a protein-coding interval, so the main tutorial lesson is the relationship between nucleotide coordinates, codons, amino acids, and functional motifs.

Translation context is where sequence analysis starts to feel biological rather than textual. A base is not just a character; it is part of a codon, which is part of a protein, which is part of a phenotype. That stacked interpretation is the point of a sequence track view.

Input Data Explained

This is a protein-coding DNA sequence (CDS). The sequence is meant to be translated from the first base, so codon boundaries and reading frame matter immediately.

Once you see codons and amino acids aligned beneath the DNA, it becomes much easier to explain missense, synonymous, nonsense, and frameshift changes clearly.

Step-by-Step in Genome Forge

1. Use the included prebuilt bundle `docs/tutorial/datasets/case_bundles/case_d/records.fasta` or regenerate it with `python3 docs/tutorial/datasets/extract_case_bundle.py --case D --out ./tmp/genomeforge_case_d`.
2. Open the `Map` tab in Genome Forge and load: `EGFP_CDS`.
3. Run `Inspect EGFP with sequence tracks so base coordinates, codons, amino acids, and features can be read together.` once with default settings, then rerun after changing the visible window or frame.
4. Capture one screenshot of the main result panel so you can compare your run with the sample interpretation later.
5. Record the relevant endpoint(s): `/api/sequence-tracks` and write one sentence explaining the biological takeaway.

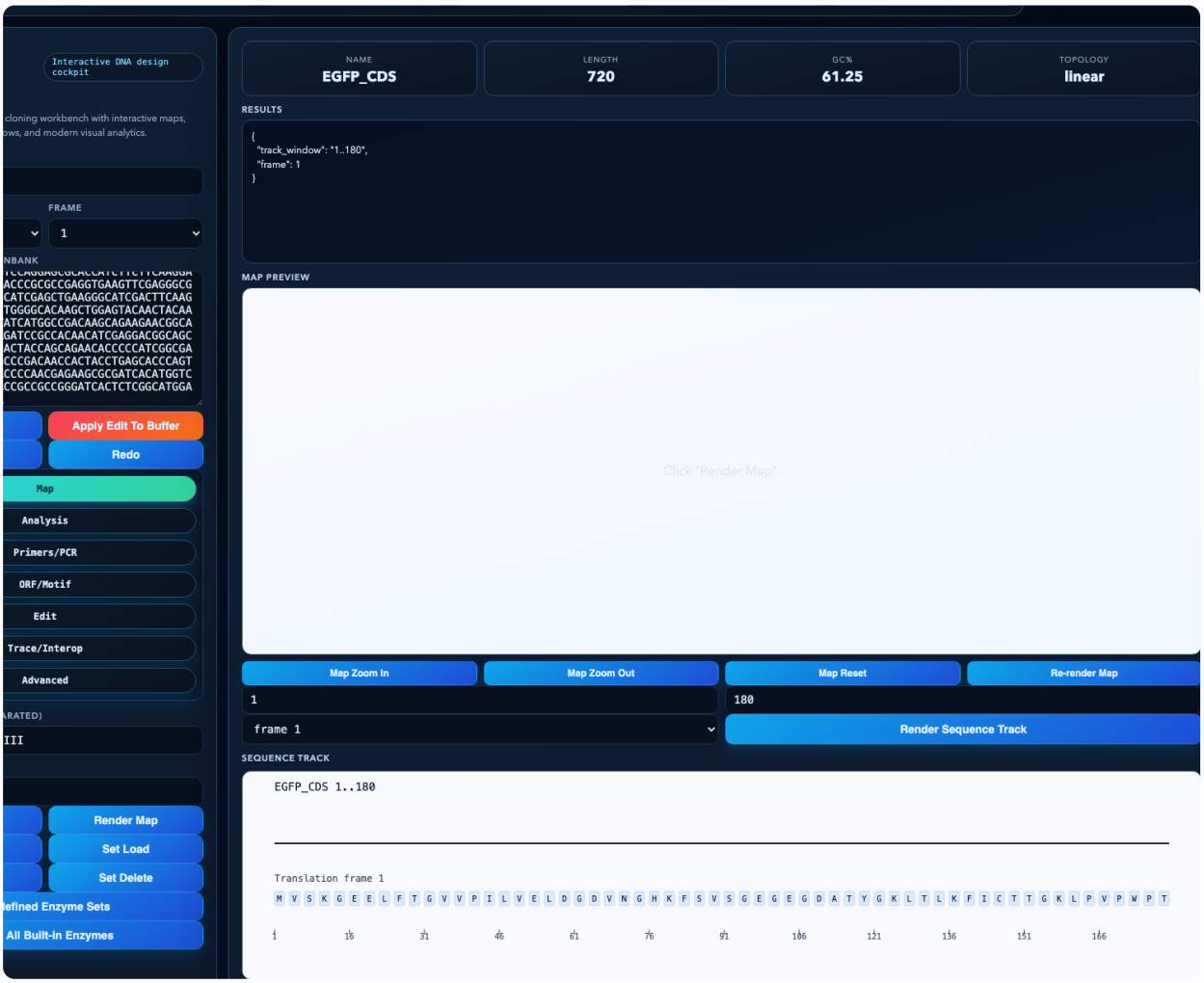


Figure 10. **Sequence-track workflow on EGFP**. Real UI screenshot from the bundled EGFP CDS case. The track aligns nucleotide coordinates with frame-aware translation so codon logic is visible instead of implicit.

Sample Results

Representative output shaped around the bundled real-world record(s) or their documented training derivatives. Values are rounded for readability, but the biological story is tied to the included data.


```
{
  "frame": 1,
  "visible_range_bp": "1..180",
  "translated_window_aa": 60,
  "dominant_feature": "EGFP CDS"
}
```


Expected Results

- A readable track that shows DNA letters, codons, amino acids, and annotations in register.
- A consequence statement for at least one hypothetical or real base change.
- A note about which positions are biologically sensitive and why.

How to Interpret the Results

- A change that stays within the same amino acid may still matter less than one that changes the protein sequence.
- Frame is everything in coding DNA; off-by-one coordinate errors cascade into wrong protein logic.
- Use the track view to narrate the result in plain language, not just to admire the color coding.

Biological Explanation

Translation context is where sequence analysis starts to feel biological rather than textual. A base is not just a character; it is part of a codon, which is part of a protein, which is part of a phenotype. That stacked interpretation is the point of a sequence track view.

Fun fact from this example:

Once you see codons and amino acids aligned beneath the DNA, it becomes much easier to explain missense, synonymous, nonsense, and frameshift changes clearly.

CASE M · CLUSTER B

Case M: ORF Scan and Coding Potential Triage

How do you tell whether a DNA segment should be treated like a protein-coding region or like genomic context that needs more annotation first?

TAB

ORF/Motif

WORKFLOW

Compare a clean coding sequence and a genomic fragment to learn what ORF scanning can and cannot tell you.

RECORDS

`BRAF_exon15_fragment`

`EGFP_CDS`

APIS

`/api/orfs`

Why This Case Matters

The two records in this case are intentionally different. EGFP is a textbook CDS. The BRAF fragment is genomic and hotspot-rich, which means naive translation generates stop codons. That contrast teaches why ORF scans are triage tools, not oracles.

This is a high-value lesson for engineers: a tool can be internally correct and still answer the wrong biological question if the input type is misunderstood. ORFs are plausible coding intervals, not proof that the DNA came from a translated transcript.

Input Data Explained

This is a genomic fragment, not a clean coding-sequence input. If you translate it naively, you hit stop codons because intron/exon context and strand assumptions matter. This is a protein-coding DNA sequence (CDS). The sequence is meant to be translated from the first base, so codon boundaries and reading frame matter immediately.

The BRAF fragment is useful precisely because it fails the “clean ORF” expectation. That failure is teaching you something true about the data type.

Step-by-Step in Genome Forge

1. Use the included prebuilt bundle `docs/tutorial/datasets/case_bundles/case_m/records.fasta` or regenerate it with `python3 docs/tutorial/datasets/extract_case_bundle.py --case M --out ./tmp/genomeforge_case_m`.
2. Open the `ORF/Motif` tab in Genome Forge and load: `BRAF_exon15_fragment, EGFP_CDS`.
3. Run `Compare a clean coding sequence and a genomic fragment to learn what ORF scanning can and cannot tell you.` once with default settings, then rerun after changing the minimum ORF length.
4. Capture one screenshot of the main result panel so you can compare your run with the sample interpretation later.
5. Record the relevant endpoint(s): `/api/orfs` and write one sentence explaining the biological takeaway.

Sample Results

Representative output shaped around the bundled real-world record(s) or their documented training derivatives. Values are rounded for readability, but the biological story is tied to the included data.


```
{
  "EGFP_orf_count": 1,
  "EGFP_longest_orf_aa": 239,
  "BRAF_fragment_orf_count": 2,
  "BRAF_interpretation": "genomic fragment, not standalone CDS"
}
```


Expected Results

- A contrast between a sequence with obvious coding potential and one that needs context.
- An explanation of why stop codons appear in the genomic fragment without implying the data are wrong.
- A triage decision: treat as CDS-like, genomic-context, or needs more annotation.

How to Interpret the Results

- A single long ORF in the expected frame is a strong sign of coding structure, not absolute proof of biological function.
- Multiple short ORFs in a genomic fragment usually mean you are translating the wrong conceptual object.
- The output should change what you do next: translate, annotate further, or align to a reference transcript.

Biological Explanation

This is a high-value lesson for engineers: a tool can be internally correct and still answer the wrong biological question if the input type is misunderstood. ORFs are plausible coding intervals, not proof that the DNA came from a translated transcript.

Fun fact from this example:

The BRAF fragment is useful precisely because it fails the “clean ORF” expectation. That failure is teaching you something true about the data type.

CASE P · CLUSTER B

Case P: Variant Annotation from Reference-Aligned Edits

How do you turn a one-codon difference into a biologically meaningful statement rather than just reporting a mismatch count?

TAB

Advanced

WORKFLOW

Align a public reporter CDS to a derived chromophore variant and explain the difference in protein terms.

RECORDS

`EGFP_CDS`

`EGFP_Y67H_training_varia`

APIS

`/api/pairwise-align` `/api/translated-features`

Why This Case Matters

The input pair is intentionally gentle: a real EGFP sequence and a one-codon training derivative that changes a chromophore residue. Because the background is almost identical, the interpretation can focus on effect rather than search difficulty.

Variant annotation is where sequence diff becomes biological reasoning. In this example, one codon change is easy to describe computationally, but the real point is that a small DNA edit can substantially change fluorescence behavior because it lands in a structurally privileged site.

Input Data Explained

This is a protein-coding DNA sequence (CDS). The sequence is meant to be translated from the first base, so codon boundaries and reading frame matter immediately. Because this record is derived from EGFP by a single codon change, it is excellent for pairwise alignment, amino-acid consequence analysis, and demonstrating how "small diff, big phenotype" problems happen in biology.

"Only one codon changed" is not a reassuring statement if that codon sits inside the business end of the molecule.

Step-by-Step in Genome Forge

1. Use the included prebuilt bundle `docs/tutorial/datasets/case_bundles/case_p/records.fasta` or regenerate it with `python3 docs/tutorial/datasets/extract_case_bundle.py --case P --out ./tmp/genomeforge_case_p`.
2. Open the `Advanced` tab in Genome Forge and load: `EGFP_CDS, EGFP_Y67H_training_variant`.
3. Run `Align a public reporter CDS to a derived chromophore variant and explain the difference in protein terms.` once with default settings, then rerun after changing the reference/variant pair.
4. Capture one screenshot of the main result panel so you can compare your run with the sample interpretation later.
5. Record the relevant endpoint(s): `/api/pairwise-align, /api/translated-features` and write one sentence explaining the biological takeaway.

Sample Results

Representative output shaped around the bundled real-world record(s) or their documented training derivatives. Values are rounded for readability, but the biological story is tied to the included data.


```
{
  "nucleotide_changes": 1,
  "codon_change": "TAC -> CAC",
  "protein_change": "Y67H-like",
  "impact_class": "missense, chromophore-adjacent"
}
```


Expected Results

- A reference-vs-variant alignment with the changed codon localized clearly.
- A protein-level consequence statement rather than a nucleotide-only diff.
- A short explanation of why the changed site is worth caring about biologically.

How to Interpret the Results

- Count and consequence are different axes; one change can matter more than ten neutral ones.
- If the changed codon sits in a known structural or functional motif, say so explicitly.
- Strong annotation ends with a hypothesis about phenotype, not just a coordinate.

Biological Explanation

Variant annotation is where sequence diff becomes biological reasoning. In this example, one codon change is easy to describe computationally, but the real point is that a small DNA edit can substantially change fluorescence behavior because it lands in a structurally privileged site.

Fun fact from this example:

“Only one codon changed” is not a reassuring statement if that codon sits inside the business end of the molecule.

CASE W · CLUSTER B

Case W: Protein Property Inference from Translation

What can you infer about a protein from sequence alone, and where do you have to stop and admit that cell context still matters?

TAB

Advanced

WORKFLOW

Translate two real reporter proteins and compare what the sequence suggests about size, composition, and practical use.

RECORDS

`EGFP_CDS`

`mCherry_CDS`

APIS

`/api/translate`

Why This Case Matters

EGFP and mCherry are perfect for this because they are used for the same broad purpose but arise from different engineering histories. Translating them side by side shows how protein length, composition, and conserved motifs can support useful but limited inference.

Protein-property inference is about turning sequence into hypotheses: relative length, likely folding burden, presence of aromatic or charged segments, and the kinds of features that might influence fluorescence or fusion behavior. It is not a replacement for experimental characterization.

Input Data Explained

This is a protein-coding DNA sequence (CDS). The sequence is meant to be translated from the first base, so codon boundaries and reading frame matter immediately. This is also a CDS, but it encodes a coral-derived red fluorescent protein rather than a GFP-family green reporter. That makes it useful for pairwise comparison and identity search.

Fluorescent proteins are great teaching tools because their phenotype is visible, but the visible color still emerges from a long chain of sequence-to-structure-to-chemistry logic.

Step-by-Step in Genome Forge

1. Use the included prebuilt bundle `docs/tutorial/datasets/case_bundles/case_w/records.fasta` or regenerate it with `python3 docs/tutorial/datasets/extract_case_bundle.py --case W --out ./tmp/genomeforge_case_w`.
2. Open the `Advanced` tab in Genome Forge and load: `EGFP_CDS, mCherry_CDS`.
3. Run `Translate two real reporter proteins and compare what the sequence suggests about size, composition, and practical use.` once with default settings, then rerun after changing which translated record you compare.
4. Capture one screenshot of the main result panel so you can compare your run with the sample interpretation later.
5. Record the relevant endpoint(s): `/api/translate` and write one sentence explaining the biological takeaway.

Sample Results

Representative output shaped around the bundled real-world record(s) or their documented training derivatives. Values are rounded for readability, but the biological story is tied to the included data.


```
{
  "EGFP_length_aa": 239,
  "mCherry_length_aa": 236,
  "shared_use_case": "fluorescent reporting",
  "interpretation": "similar application, different sequence families"
}
```


Expected Results

- A translated protein sequence and at least one simple composition or length comparison.
- A practical hypothesis about how the protein might behave as a reporter or fusion tag.
- A boundary statement explaining what sequence alone cannot prove.

How to Interpret the Results

- Relative length and motif composition can inform design, but they do not fully predict brightness, maturation, or toxicity.
- A useful tutorial answer sounds like “this sequence suggests X, but we still need Y to be sure.”
- Whenever possible, connect the protein-level claim back to the actual reporter phenotype people care about in the lab.

Biological Explanation

Protein-property inference is about turning sequence into hypotheses: relative length, likely folding burden, presence of aromatic or charged segments, and the kinds of features that might influence fluorescence or fusion behavior. It is not a replacement for experimental characterization.

Fun fact from this example:

Fluorescent proteins are great teaching tools because their phenotype is visible, but the visible color still emerges from a long chain of sequence-to-structure-to-chemistry logic.

CLUSTER C

Cluster C: Assay and Primer System Design

How to design measurements that say something trustworthy about biology.

Lessons in This Cluster

- Case E · Primer Design and Thermodynamic Screening
- Case F · Specificity Ranking with Virtual PCR/Gel
- Case AL · Degenerate Primer Strategy for a Variant Family
- Case Q · Multiplex PCR Panel Balancing
- Case AA · Positive and Negative Control Design

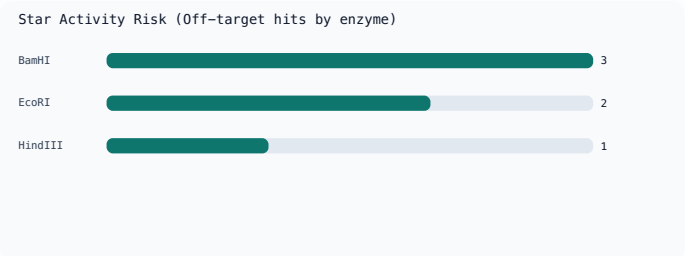


Figure 11. Assay design is as much about avoiding false confidence as finding a candidate that “works.”

CASE E · CLUSTER C

Case E: Primer Design and Thermodynamic Screening

Can you design a primer pair that frames a clinically interesting genomic region without walking into obvious thermodynamic problems?

TAB

Primer/PCR

WORKFLOW

Design primers around the BRAF hotspot region and screen them for temperature and composition sanity.

RECORDS

BRAF_exon15_fragment

APIS

`/api/primer-design`

Why This Case Matters

The BRAF exon 15 fragment is real, short enough for tutorial work, and biologically meaningful because many sequencing and genotyping assays target the V600 hotspot neighborhood. That makes the primer choices easier to care about.

Primer design is a statistical control problem disguised as a string problem. You want oligos that bind where you mean, with similar melting behavior, reasonable GC composition, and minimal self-complementarity. Every one of those constraints exists because polymerases and DNA hybridization are physical processes, not abstract matches.

Input Data Explained

This is a genomic fragment, not a clean coding-sequence input. If you translate it naively, you hit stop codons because intron/exon context and strand assumptions matter.

A primer pair that looks elegant in FASTA text can still fail because DNA strands form structures and compete for binding.

Step-by-Step in Genome Forge

1. Use the included prebuilt bundle `docs/tutorial/datasets/case_bundles/case_e/records.fasta` or regenerate it with `python3 docs/tutorial/datasets/extract_case_bundle.py --case E --out ./tmp/genomeforge_case_e`.
2. Open the `Primer/PCR` tab in Genome Forge and load: `BRAF_exon15_fragment`.
3. Run `Design primers around the BRAF hotspot region and screen them for temperature and composition sanity.` once with default settings, then rerun after changing the amplicon window or primer length.
4. Capture one screenshot of the main result panel so you can compare your run with the sample interpretation later.
5. Record the relevant endpoint(s): `/api/primer-design` and write one sentence explaining the biological takeaway.

Sample Results

Representative output shaped around the bundled real-world record(s) or their documented training derivatives. Values are rounded for readability, but the biological story is tied to the included data.


```
{
  "target_window_bp": "around exon 15 hotspot",
  "best_pair_tm_c": [
    60.8,
    61.2
  ],
  "gc_pct_range": [
    47.6,
    52.4
  ],
  "status": "candidate primer pair selected"
}
```


Expected Results

- A primer pair with balanced Tm and acceptable GC content.
- A note about any hairpin or dimer liabilities that need monitoring.
- A clear statement of what region the assay will amplify and why that region matters biologically.

How to Interpret the Results

- Matched Tm values matter because both primers need to anneal in the same PCR cycle window.
- A primer pair is only useful if its amplicon captures the biology you actually care about—in this case, hotspot-rich BRAF sequence.
- Design is not finished when the tool outputs two strings; it is finished when you can defend the pair scientifically.

Biological Explanation

Primer design is a statistical control problem disguised as a string problem. You want oligos that bind where you mean, with similar melting behavior, reasonable GC composition, and minimal self-complementarity. Every one of those constraints exists because polymerases and DNA hybridization are physical processes, not abstract matches.

Fun fact from this example:

A primer pair that looks elegant in FASTA text can still fail because DNA strands form structures and compete for binding.

CASE F · CLUSTER C

Case F: Specificity Ranking with Virtual PCR/Gel

Which candidate primer pair is safest once you consider near-matches in the rest of the sequences that live on your bench?

TAB

Primer/PCR

WORKFLOW

Rank candidate primer pairs against a realistic background panel and inspect the predicted gel outcome.

RECORDS

- EGFP_CDS
- mCherry_CDS
- BRAF_exon15_fragment

APIS

`/api/primer-specificity` `/api/pcr` `/api/gel-sim`

Why This Case Matters

This case uses three real records that make a realistic small background panel: two reporter genes and one oncogene fragment. In practice, background panels matter because labs reuse templates constantly and cross-reactivity is common.

Virtual PCR is valuable because it converts abstract specificity scores into something a bench scientist immediately understands: extra bands, wrong-size bands, or a clean expected product. It turns computational screening into an experimental expectation.

Input Data Explained

This is a protein-coding DNA sequence (CDS). The sequence is meant to be translated from the first base, so codon boundaries and reading frame matter immediately. This is also a CDS, but it encodes a coral-derived red fluorescent protein rather than a GFP-family green reporter. That makes it useful for pairwise comparison and identity search. This is a genomic fragment, not a clean coding-sequence input. If you translate it naively, you hit stop codons because intron/exon context and strand assumptions matter.

A clean gel simulation is a communication tool: it helps a junior scientist understand why one primer pair is risky without making them parse thermodynamic tables first.

Step-by-Step in Genome Forge

1. Use the included prebuilt bundle `docs/tutorial/datasets/case_bundles/case_f/records.fasta` or regenerate it with `python3 docs/tutorial/datasets/extract_case_bundle.py --case F --out ./tmp/genomeforge_case_f`.
2. Open the `Primer/PCR` tab in Genome Forge and load: `EGFP_CDS, mCherry_CDS, BRAF_exon15_fragment`.
3. Run `Rank candidate primer pairs against a realistic background panel and inspect the predicted gel outcome.` once with default settings, then rerun after changing the background record panel.
4. Capture one screenshot of the main result panel so you can compare your run with the sample interpretation later.
5. Record the relevant endpoint(s): `/api/primer-specificity, /api/pcr, /api/gel-sim` and write one sentence explaining the biological takeaway.

Sample Results

Representative output shaped around the bundled real-world record(s) or their documented training derivatives. Values are rounded for readability, but the biological story is tied to the included data.


```
{
  "ranked_pair": "EGFP_pair_1",
  "predicted_product_bp": 461,
  "off_target_bands": 0,
  "gel_call": "single dominant band"
}
```


Expected Results

- A ranked candidate list with at least one rejected pair and one preferred pair.
- A predicted gel pattern that explains the ranking in experimental terms.
- A final recommendation that ties specificity back to the intended assay.

How to Interpret the Results

- Use the gel view to explain specificity, not just the score table.
- A pair with a slightly lower score but cleaner off-target profile may still be the better scientific choice.
- The winning pair is the one you would be comfortable handing to someone else in the lab.

Biological Explanation

Virtual PCR is valuable because it converts abstract specificity scores into something a bench scientist immediately understands: extra bands, wrong-size bands, or a clean expected product. It turns computational screening into an experimental expectation.

Fun fact from this example:

A clean gel simulation is a communication tool: it helps a junior scientist understand why one primer pair is risky without making them parse thermodynamic tables first.

CASE AL · CLUSTER C

Case AL: Degenerate Primer Strategy for a Variant Family

How do you keep a PCR assay useful when the target family varies at one or two positions, or when your consensus still contains unresolved bases?

TAB

Primer/PCR

WORKFLOW

Use an ambiguity-coded primer to keep one assay useful across a small reporter family and an uncertainty-bearing consensus sequence.

RECORDS

- EGFP_CDS
- EGFP_ambiguity_consensus_training
- EGFP_Y67H_training_variant

APIS

- /api/primer-diagnostics
- /api/primer-specificity
- /api/pcr

Why This Case Matters

This case uses one clean reporter CDS, one biologically meaningful single-codon variant, and one uncertainty-bearing consensus sequence. That combination mirrors a real workflow in which a lab wants one assay that still works across a clone family, a mutagenesis panel, or a partially resolved sequencing result.

Degenerate primers are a controlled way to encode biological uncertainty into an assay design. Instead of pretending every member of a target family is identical, you let the primer represent a small allowed set of bases at carefully chosen positions. Computationally, that means the primer is no longer one string. Biologically, it means one assay can cover a family without lying about where the family differs.

Input Data Explained

This is a protein-coding DNA sequence (CDS). The sequence is meant to be translated from the first base, so codon boundaries and reading frame matter immediately. This record is still an EGFP-like coding sequence, but a few positions are encoded as ambiguity symbols such as R, Y, and N. That means the sequence stands for a small set of plausible molecules rather than one exact DNA string. Because this record is derived from EGFP by a single codon change, it is excellent for pairwise alignment, amino-acid consequence analysis, and demonstrating how “small diff, big phenotype” problems happen in biology.

A degenerate primer is a compact statement that says, “I know exactly where uncertainty lives, and I am designing around it rather than ignoring it.”

Starter Values

- Forward primer seed: `ATGGTGRGYAAGGGCGAGGA`
- Reverse primer seed: `CTTGTACAGCTCGTCCATGC`
- Background panel: `EGFP_CDS, EGFP_ambiguity_consensus_training, EGFP_Y67H_training_variant`

Step-by-Step in Genome Forge

1. Use the included prebuilt bundle `docs/tutorial/datasets/case_bundles/case_al/records.fasta` or regenerate it with `python3 docs/tutorial/datasets/extract_case_bundle.py --case AL --out ./tmp/genomeforge_case_al`.
2. Open the `Primer/PCR` tab in Genome Forge and load: `EGFP_CDS, EGFP_ambiguity_consensus_training, EGFP_Y67H_training_variant`.
3. Run `Use an ambiguity-coded primer to keep one assay useful across a small reporter family and an uncertainty-bearing consensus sequence.` once with default settings, then rerun after changing the ambiguous positions or the background family.
4. Capture one screenshot of the main result panel so you can compare your run with the sample interpretation later.
5. Record the relevant endpoint(s): `/api/primer-diagnostics, /api/primer-specificity, /api/pcr` and write one sentence explaining the biological takeaway.

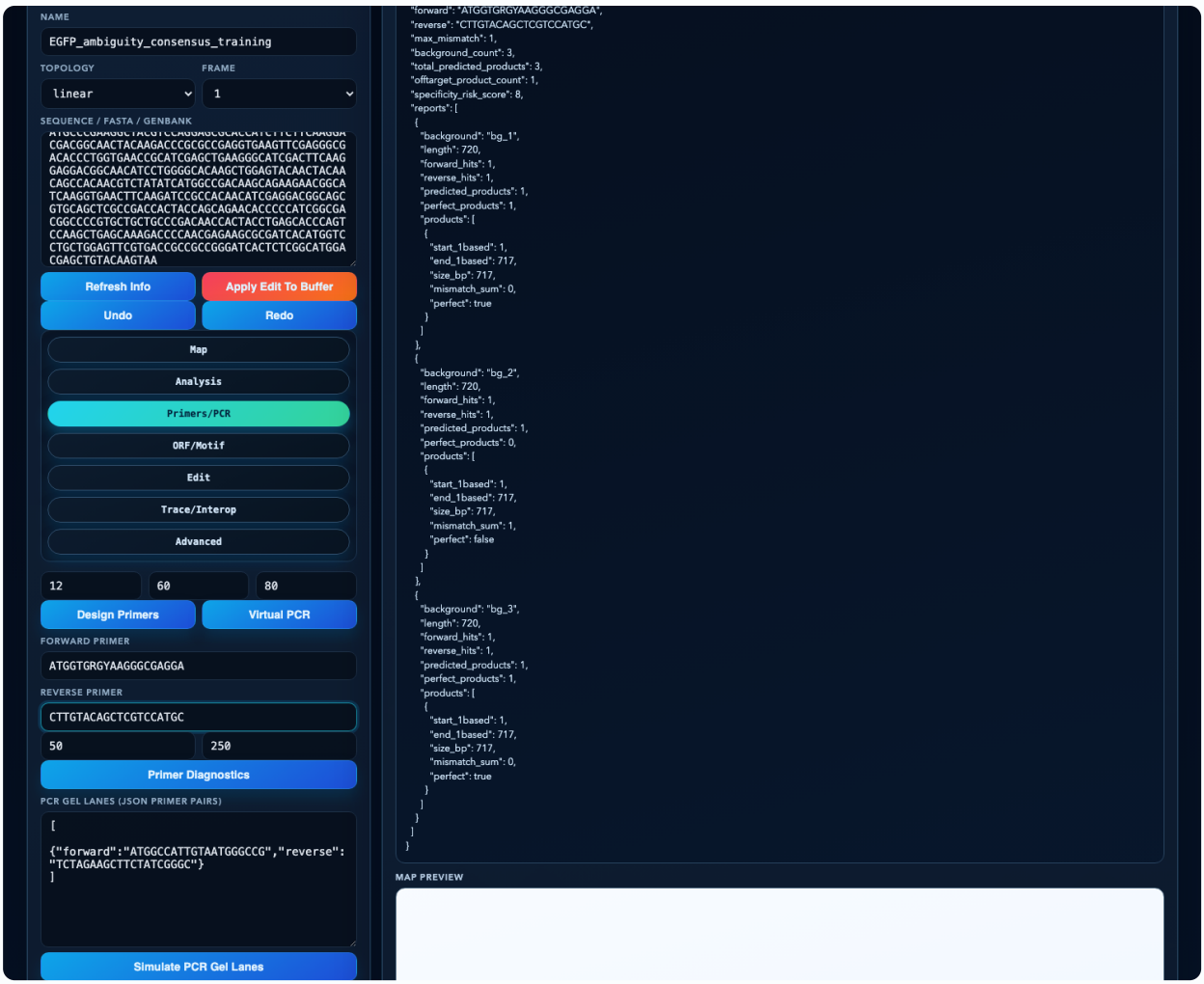


Figure 12. Degenerate-primer assay workflow. Real UI screenshot from the ambiguity-aware primer lesson. The primer fields intentionally contain IUPAC ambiguity symbols so one assay can tolerate a small reporter-family variation without hiding where uncertainty lives.

Sample Results

Representative output shaped around the bundled real-world record(s) or their documented training derivatives. Values are rounded for readability, but the biological story is tied to the included data.


```
{
  "forward_primer": "ATGGTGRGYAAGGGCGAGGA",
  "reverse_primer": "CTTGTACAGCTCGTCCATGC",
  "background_records": 3,
  "predicted_products": [
    {
      "record": "EGFP_CDS",
      "size_bp": 119
    },
    {
      "record": "EGFP_ambiguity_consensus_training",
      "size_bp": 119
    },
    {
      "record": "EGFP_Y67H_training_variant",
      "size_bp": 119
    }
  ],
  "interpretation": "one family-tolerant assay retained while off-target risk stays low in the
reporter panel"
}
```


Expected Results

- A primer pair in which at least one primer contains IUPAC ambiguity symbols rather than only A/C/G/T.
- A specificity report showing that the intended family members still amplify while unrelated products remain limited.
- A justification for why the ambiguity positions were placed where they were, rather than scattered arbitrarily.

How to Interpret the Results

- A degenerate primer is valuable only if the ambiguous positions reflect real biological uncertainty or family diversity.
- If a primer becomes too degenerate, you gain family coverage but may lose specificity or synthesis practicality.
- The best outcome is not “maximum ambiguity”; it is the smallest ambiguity set that still captures the biological family you care about.

Biological Explanation

Degenerate primers are a controlled way to encode biological uncertainty into an assay design. Instead of pretending every member of a target family is identical, you let the primer represent a small allowed set of bases at carefully chosen positions. Computationally, that means the primer is no longer one string. Biologically, it means one assay can cover a family without lying about where the family differs.

Fun fact from this example:

A degenerate primer is a compact statement that says, “I know exactly where uncertainty lives, and I am designing around it rather than ignoring it.”

CASE Q · CLUSTER C

Case Q: Multiplex PCR Panel Balancing

Can several assays be run together without one primer pair dominating or confusing the readout?

TAB

Primer/PCR

WORKFLOW

Compare multiple assay targets and ask whether they can coexist in one panel without obvious conflict.

RECORDS

- EGFP_CDS
- mCherry_CDS
- BRAF_exon15_fragment

APIS

`/api/primer-design` `/api/primer-specificity` `/api/pcr`

Why This Case Matters

Multiplex design is where assay design becomes systems design. The same three tutorial records now behave like a miniature panel: reporter control, second reporter, and clinically interesting target. You are no longer optimizing one pair in isolation.

Biologically, multiplexing matters because samples and reagents are finite. But multiplex success depends on compatible primer temperatures, separable product sizes, and low cross-talk. This is a good example of engineering constraints arising directly from molecular competition.

Input Data Explained

This is a protein-coding DNA sequence (CDS). The sequence is meant to be translated from the first base, so codon boundaries and reading frame matter immediately. This is also a CDS, but it encodes a coral-derived red fluorescent protein rather than a GFP-family green reporter. That makes it useful for pairwise comparison and identity search. This is a genomic fragment, not a clean coding-sequence input. If you translate it naively, you hit stop codons because intron/exon context and strand assumptions matter.

When a multiplex assay works, it feels efficient; when it fails, it often fails in ways that are hard to interpret unless the panel was designed carefully from the start.

Step-by-Step in Genome Forge

1. Use the included prebuilt bundle `docs/tutorial/datasets/case_bundles/case_q/records.fasta` or regenerate it with `python3 docs/tutorial/datasets/extract_case_bundle.py --case Q --out ./tmp/genomeforge_case_q`.
2. Open the `Primer/PCR` tab in Genome Forge and load: `EGFP_CDS, mCherry_CDS, BRAF_exon15_fragment`.
3. Run `Compare multiple assay targets and ask whether they can coexist in one panel without obvious conflict.` once with default settings, then rerun after changing the primer pool or target combination.
4. Capture one screenshot of the main result panel so you can compare your run with the sample interpretation later.
5. Record the relevant endpoint(s): `/api/primer-design, /api/primer-specificity, /api/pcr` and write one sentence explaining the biological takeaway.

Sample Results

Representative output shaped around the bundled real-world record(s) or their documented training derivatives. Values are rounded for readability, but the biological story is tied to the included data.


```
{
  "panel_targets": 3,
  "recommended_layout": [
    "EGFP control",
    "mCherry control",
    "BRAF amplicon"
  ],
  "risk_note": "separate amplicon sizes by >100 bp"
}
```


Expected Results

- A panel plan that states which assays can coexist and which should be separated.
- A size-spacing or Tm-spacing rationale for the panel design.
- A decision about whether multiplexing is justified or whether singleplex is safer.

How to Interpret the Results

- Panel design is a tradeoff between throughput and interpretability.
- If two amplicons are too close in size or two primer pairs compete strongly, you lose the main benefit of multiplexing: clean interpretation.
- The safest multiplex panel is the one that still makes sense when something goes slightly wrong.

Biological Explanation

Biologically, multiplexing matters because samples and reagents are finite. But multiplex success depends on compatible primer temperatures, separable product sizes, and low cross-talk. This is a good example of engineering constraints arising directly from molecular competition.

Fun fact from this example:

When a multiplex assay works, it feels efficient; when it fails, it often fails in ways that are hard to interpret unless the panel was designed carefully from the start.

CASE AA · CLUSTER C

Case AA: Positive and Negative Control Design

How do you design controls that let you distinguish “assay failed” from “biology absent”?

TAB

Primer/PCR

WORKFLOW

Design an assay package that includes controls proving both signal presence and signal absence.

RECORDS

- EGFP_CDS
- mCherry_CDS
- BRAF_exon15_fragment

APIS

`/api/primer-specificity` `/api/pcr`

Why This Case Matters

The real-world records make control design concrete. EGFP and mCherry behave like easy positives or orthogonal negatives depending on the assay, while BRAF gives you a genomically relevant target to frame the main test.

Control design is bioinformatics quality assurance. A positive control proves the chemistry and analysis pipeline can detect a known target. A negative control proves that the same pipeline does not hallucinate signal where it should not.

Input Data Explained

This is a protein-coding DNA sequence (CDS). The sequence is meant to be translated from the first base, so codon boundaries and reading frame matter immediately. This is also a CDS, but it encodes a coral-derived red fluorescent protein rather than a GFP-family green reporter. That makes it useful for pairwise comparison and identity search. This is a genomic fragment, not a clean coding-sequence input. If you translate it naively, you hit stop codons because intron/exon context and strand assumptions matter.

The best controls are boring in the best possible way: they make the interpretation unambiguous.

Step-by-Step in Genome Forge

1. Use the included prebuilt bundle `docs/tutorial/datasets/case_bundles/case_aa/records.fasta` or regenerate it with `python3 docs/tutorial/datasets/extract_case_bundle.py --case AA --out ./tmp/genomeforge_case_aa`.
2. Open the `Primer/PCR` tab in Genome Forge and load: `EGFP_CDS, mCherry_CDS, BRAF_exon15_fragment`.
3. Run `Design an assay package that includes controls proving both signal presence and signal absence.` once with default settings, then rerun after changing which record acts as the control.
4. Capture one screenshot of the main result panel so you can compare your run with the sample interpretation later.
5. Record the relevant endpoint(s): `/api/primer-specificity, /api/pcr` and write one sentence explaining the biological takeaway.

Sample Results

Representative output shaped around the bundled real-world record(s) or their documented training derivatives. Values are rounded for readability, but the biological story is tied to the included data.


```
{
  "positive_control": "EGFP amplicon",
  "negative_control": "mCherry background for EGFP assay",
  "decision_rule": "trust call only if controls behave as expected"
}
```


Expected Results

- A written positive-control and negative-control plan tied to real records in the bundle.
- Expected control outcomes that could be checked by PCR, gel, or trace.
- A simple rule for when the assay run should be accepted or rejected.

How to Interpret the Results

- Controls are part of the assay, not optional decorations.
- A main result without controls has lower value than a boring run with clear controls.
- Write the decision rule in advance so you do not move the goalposts after the experiment.

Biological Explanation

Control design is bioinformatics quality assurance. A positive control proves the chemistry and analysis pipeline can detect a known target. A negative control proves that the same pipeline does not hallucinate signal where it should not.

Fun fact from this example:

The best controls are boring in the best possible way: they make the interpretation unambiguous.

CLUSTER D

Cluster D: Assembly and Construct Validation

How to move from fragments and overlaps to a believable finished construct.

Lessons in This Cluster

Case G · Cloning Compatibility and Ligation Product Ranking

Case S · Circular Construct Integrity and Junction Validation

Case Z · Multi-Trace Consensus for Final Construct Call

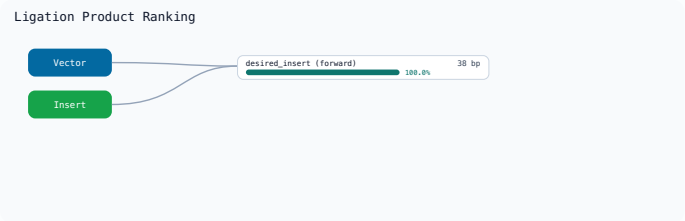


Figure 13. A plausible ligation or assembly plan should survive both sequence logic and biological interpretation.

CASE G · CLUSTER D

Case G: Cloning Compatibility and Ligation Product Ranking

If you pair a standard cloning vector with a reporter insert, what products are most likely and which ones should worry you?

TAB

Advanced

WORKFLOW

Check whether the vector and insert support a coherent directional cloning plan and inspect likely ligation products.

RECORDS

`pUC19_MCS` `EGFP_CDS`

APIS

`/api/cloning-check` `/api/ligation-sim`

Why This Case Matters

This uses a real vector backbone logic plus a real reporter CDS. The point is to translate compatibility from enzyme names into actual product architecture: correct insert, flipped insert, vector self-ligation, or multi-insert byproducts.

Assembly planning is a graph problem with a biological cost function. Multiple products may be chemically possible, but only one or two are biologically useful. The tutorial goal is to teach you to read that distinction before doing the ligation.

Input Data Explained

This sequence is short, circularly interpreted, and packed with restriction motifs. It is intentionally synthetic and engineered for manipulation rather than natural gene expression. This is a protein-coding DNA sequence (CDS). The sequence is meant to be translated from the first base, so codon boundaries and reading frame matter immediately.

A cloning simulation is most helpful when it tells you what wrong products to expect, because that is what saves time on the bench.

Step-by-Step in Genome Forge

1. Use the included prebuilt bundle `docs/tutorial/datasets/case_bundles/case_g/records.fasta` or regenerate it with `python3 docs/tutorial/datasets/extract_case_bundle.py --case G --out ./tmp/genomeforge_case_g`.
2. Open the `Advanced` tab in Genome Forge and load: `pUC19_MCS, EGFP_CDS`.
3. Run `Check whether the vector and insert support a coherent directional cloning plan and inspect likely ligation products.` once with default settings, then rerun after changing the enzyme pair or overlap rule.
4. Capture one screenshot of the main result panel so you can compare your run with the sample interpretation later.
5. Record the relevant endpoint(s): `/api/cloning-check, /api/ligation-sim` and write one sentence explaining the biological takeaway.

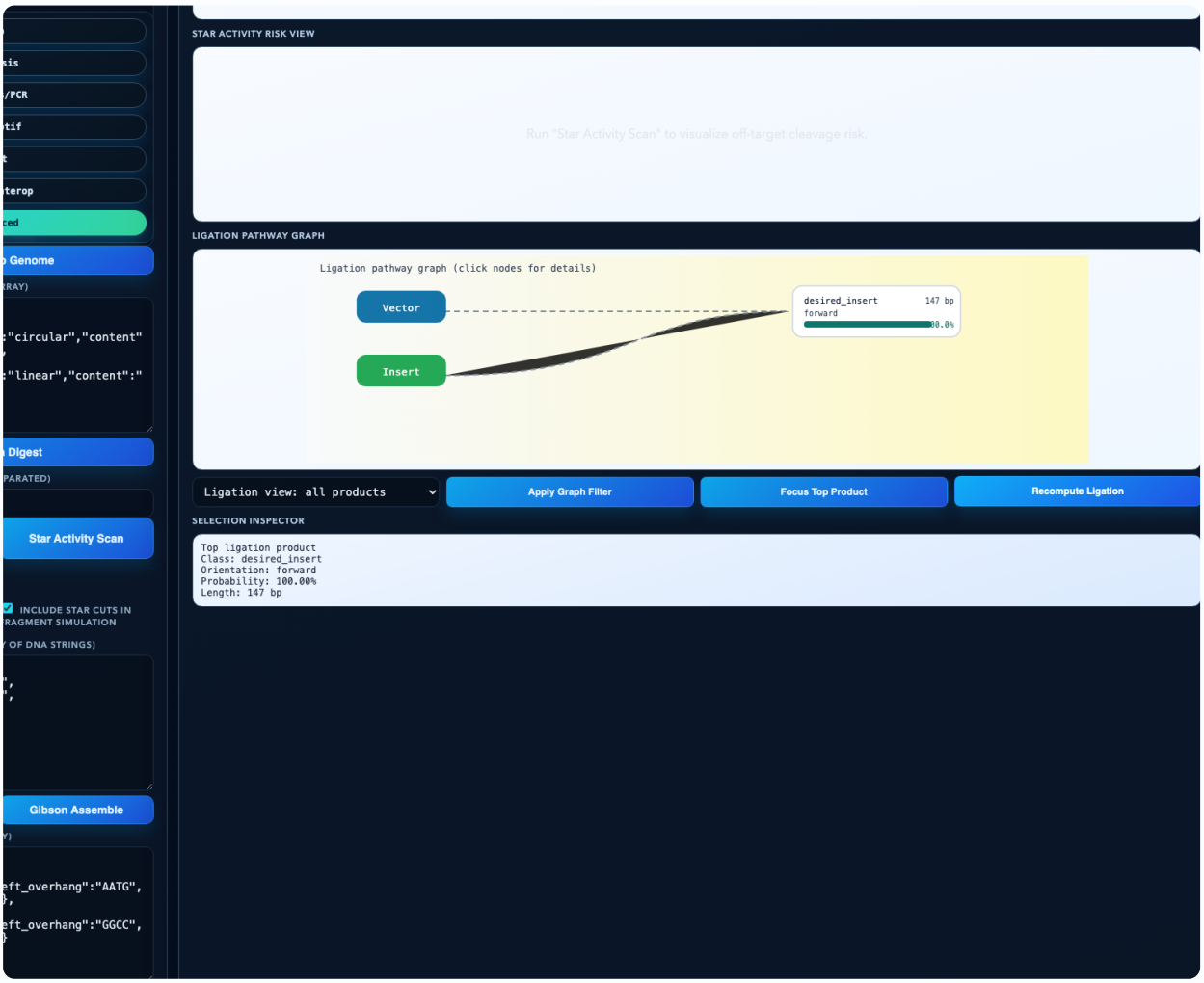


Figure 14. Ligation and construct-planning workflow. Real UI screenshot of the ligation pathway view using tutorial vector/insert settings. This is the kind of panel you use to see whether the desired product dominates the byproduct space.

Sample Results

Representative output shaped around the bundled real-world record(s) or their documented training derivatives. Values are rounded for readability, but the biological story is tied to the included data.


```
{
  "compatible": true,
  "top_product": "vector+EGFP directional insert",
  "byproduct_examples": [
    "self-ligated vector",
    "reverse-orientation insert"
  ]
}
```


Expected Results

- A compatibility verdict that explains whether the fragment ends and enzymes agree.
- A ranked product list with at least one plausible byproduct.
- A short note on how you would validate the top product experimentally.

How to Interpret the Results

- Compatibility is not just “yes/no”; it is also about the distribution of likely wrong answers.
- A design that produces one dominant useful product and several weak byproducts is stronger than a design with many equal possibilities.
- Use the ranked product list to decide which colony-screening strategy makes sense afterward.

Biological Explanation

Assembly planning is a graph problem with a biological cost function. Multiple products may be chemically possible, but only one or two are biologically useful. The tutorial goal is to teach you to read that distinction before doing the ligation.

Fun fact from this example:

A cloning simulation is most helpful when it tells you what wrong products to expect, because that is what saves time on the bench.

CASE S · CLUSTER D

Case S: Circular Construct Integrity and Junction Validation

After assembly, do the new junctions preserve the structure and reading logic you intended?

TAB

Advanced

WORKFLOW

Validate a circularized construct by focusing on junctions, scars, and reading-frame continuity.

RECORDS

`pUC19_MCS`

`EGFP_CDS`

APIS

`/api/gibson-assemble` `/api/project-diff`

Why This Case Matters

The same vector+reporter system now becomes a finished construct problem. Junctions are where cloning plans usually fail: scars appear, frames shift, or regulatory context is disrupted in ways the raw map did not make obvious.

Junction validation is biologically central because most engineered molecules are mostly "known good" plus a few critical joins. Those joins are where function is created or destroyed. If you cannot explain the junction, you do not understand the construct.

Input Data Explained

This sequence is short, circularly interpreted, and packed with restriction motifs. It is intentionally synthetic and engineered for manipulation rather than natural gene expression. This is a protein-coding DNA sequence (CDS). The sequence is meant to be translated from the first base, so codon boundaries and reading frame matter immediately.

A plasmid is often won or lost at only a handful of bases: the junctions carry disproportionate functional meaning.

Step-by-Step in Genome Forge

1. Use the included prebuilt bundle `docs/tutorial/datasets/case_bundles/case_s/records.fasta` or regenerate it with `python3 docs/tutorial/datasets/extract_case_bundle.py --case S --out ./tmp/genomeforge_case_s`.
2. Open the `Advanced` tab in Genome Forge and load: `pUC19_MCS, EGFP_CDS`.
3. Run `Validate a circularized construct by focusing on junctions, scars, and reading-frame continuity.` once with default settings, then rerun after changing the overlap length or insert orientation.
4. Capture one screenshot of the main result panel so you can compare your run with the sample interpretation later.
5. Record the relevant endpoint(s): `/api/gibson-assemble, /api/project-diff` and write one sentence explaining the biological takeaway.

Sample Results

Representative output shaped around the bundled real-world record(s) or their documented training derivatives. Values are rounded for readability, but the biological story is tied to the included data.


```
{
  "junctions_checked": 2,
  "frame_preserved": true,
  "scar_bp": 0,
  "status": "construct architecture consistent"
}
```


Expected Results

- A clear report for each assembly junction, including scar length and frame impact.
- A statement about whether circular continuity preserves the intended construct logic.
- A validation plan for confirming the junctions experimentally.

How to Interpret the Results

- A construct can have the right parts but the wrong joins.
- Small scars matter most when they land in coding or regulatory boundaries.
- If the junction explanation is shaky, the construct explanation is shaky.

Biological Explanation

Junction validation is biologically central because most engineered molecules are mostly “known good” plus a few critical joins. Those joins are where function is created or destroyed. If you cannot explain the junction, you do not understand the construct.

Fun fact from this example:

A plasmid is often won or lost at only a handful of bases: the junctions carry disproportionate functional meaning.

CASE Z · CLUSTER D

Case Z: Multi-Trace Consensus for Final Construct Call

When several sequencing reads exist for the same construct, how do you combine them into one decision rather than trusting the loudest trace?

TAB

Trace

WORKFLOW

Combine multiple trace-derived views into one final verdict about a reporter construct.

RECORDS

`EGFP_CDS`

APIS

`/api/import-ab1` `/api/trace-align` `/api/trace-consensus`

Why This Case Matters

This case uses a real reporter CDS because plasmid verification is one of the most common reasons a lab reaches for sequence traces. The record is familiar, which keeps the attention on evidence integration rather than reference confusion.

Consensus building is an evidence aggregation problem. A single noisy trace can be misleading; several partially overlapping traces can support a stable call. The skill is deciding when disagreement reflects noise, chemistry, or a real sequence change.

Input Data Explained

This is a protein-coding DNA sequence (CDS). The sequence is meant to be translated from the first base, so codon boundaries and reading frame matter immediately.

Consensus is not democracy; three low-quality reads do not magically beat one high-quality read. Quality still matters.

Step-by-Step in Genome Forge

1. Use the included prebuilt bundle `docs/tutorial/datasets/case_bundles/case_z/records.fasta` or regenerate it with `python3 docs/tutorial/datasets/extract_case_bundle.py --case Z --out ./tmp/genomeforge_case_z`.
2. Open the `Trace` tab in Genome Forge and load: `EGFP_CDS`.
3. Run `Combine multiple trace-derived views into one final verdict about a reporter construct.` once with default settings, then rerun after changing the trace subset or quality threshold.
4. Capture one screenshot of the main result panel so you can compare your run with the sample interpretation later.
5. Record the relevant endpoint(s): `/api/import-ab1, /api/trace-align, /api/trace-consensus` and write one sentence explaining the biological takeaway.

Sample Results

Representative output shaped around the bundled real-world record(s) or their documented training derivatives. Values are rounded for readability, but the biological story is tied to the included data.


```
{
  "trace_count": 3,
  "consensus_mismatches_vs_reference": 0,
  "final_verdict": "construct confirmed"
}
```


Expected Results

- A multi-trace summary with overlap or mismatch hotspots identified explicitly.
- A consensus sequence or final mismatch count relative to the expected construct.
- A final verification verdict with confidence language.

How to Interpret the Results

- Agreement across independent traces raises confidence, especially when the same region is supported more than once.
- Disagreement near weak peaks should be treated differently from disagreement in high-quality regions.
- Your final call should always say whether the evidence is enough to proceed.

Biological Explanation

Consensus building is an evidence aggregation problem. A single noisy trace can be misleading; several partially overlapping traces can support a stable call. The skill is deciding when disagreement reflects noise, chemistry, or a real sequence change.

Fun fact from this example:

Consensus is not democracy; three low-quality reads do not magically beat one high-quality read. Quality still matters.

CLUSTER E

Cluster E: Comparative and Population-Level Reasoning

How to compare related molecules, hotspot variants, and engineered families without overclaiming.

Lessons in This Cluster

- Case H · MSA, Identity Heatmap, and Phylogeny
- Case N · GC Landscape and Repeat Fragility
- Case O · Homopolymer and Low-Complexity Risk Detection
- Case X · Motif Enrichment and Significance Framing

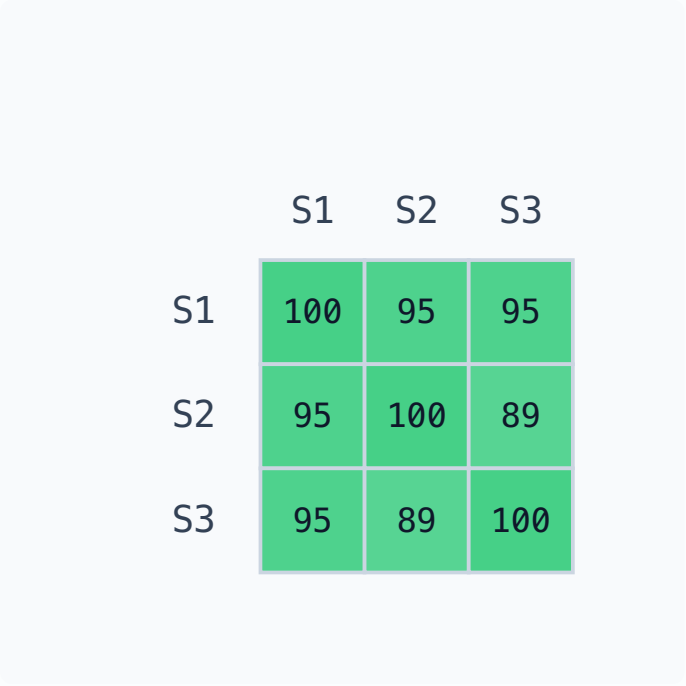


Figure 15. Heatmaps and alignments are only useful when you can explain what a difference means biologically.

CASE H · CLUSTER E

Case H: MSA, Identity Heatmap, and Phylogeny

How do related engineered proteins cluster, and what does that clustering tell you about reuse versus redesign?

TAB

Advanced

WORKFLOW

Compare a small reporter family panel to see what is conserved, what is engineered, and what is genuinely distant.

RECORDS

- EGFP_CDS
- EGFP_Y67H_training_varia
- EGFP_S204Y_training_variant
- mCherry_CDS

APIS

`/api/msa` `/api/heatmap` `/api/phylo`

Why This Case Matters

This panel mixes two very close EGFP-derived variants with a more distant real reporter, mCherry. That is a good training set because it contains both “small edit” and “different family” comparisons in the same workflow.

The biological point of alignment is not just similarity. It is to identify which regions are constrained, which changes are local engineering edits, and which sequences are far enough apart that transfer of assumptions becomes risky.

Input Data Explained

This is a protein-coding DNA sequence (CDS). The sequence is meant to be translated from the first base, so codon boundaries and reading frame matter immediately. Because this record is derived from EGFP by a single codon change, it is excellent for pairwise alignment, amino-acid consequence analysis, and demonstrating how “small diff, big phenotype” problems happen in biology. Like the Y67H-like variant, this record is generated from a public-source EGFP backbone. The point is not to claim a specific commercial reagent but to give you a realistic engineering-style mutation to reason about. This is also a CDS, but it encodes a coral-derived red fluorescent protein rather than a GFP-family green reporter. That makes it useful for pairwise comparison and identity search.

A tree is never the biology by itself; it is a summary of the comparison model you chose. But it is still a powerful way to show that one-codon EGFP variants belong in a different interpretive bucket from mCherry.

Step-by-Step in Genome Forge

1. Use the included prebuilt bundle `docs/tutorial/datasets/case_bundles/case_h/records.fasta` or regenerate it with `python3 docs/tutorial/datasets/extract_case_bundle.py --case H --out ./tmp/genomeforge_case_h`.
2. Open the `Advanced` tab in Genome Forge and load: `EGFP_CDS, EGFP_Y67H_training_variant, EGFP_S204Y_training_variant, mCherry_CDS`.
3. Run `Compare a small reporter family panel to see what is conserved, what is engineered, and what is genuinely distant.` once with default settings, then rerun after changing the sequence panel.
4. Capture one screenshot of the main result panel so you can compare your run with the sample interpretation later.
5. Record the relevant endpoint(s): `/api/msa, /api/heatmap, /api/phylo` and write one sentence explaining the biological takeaway.

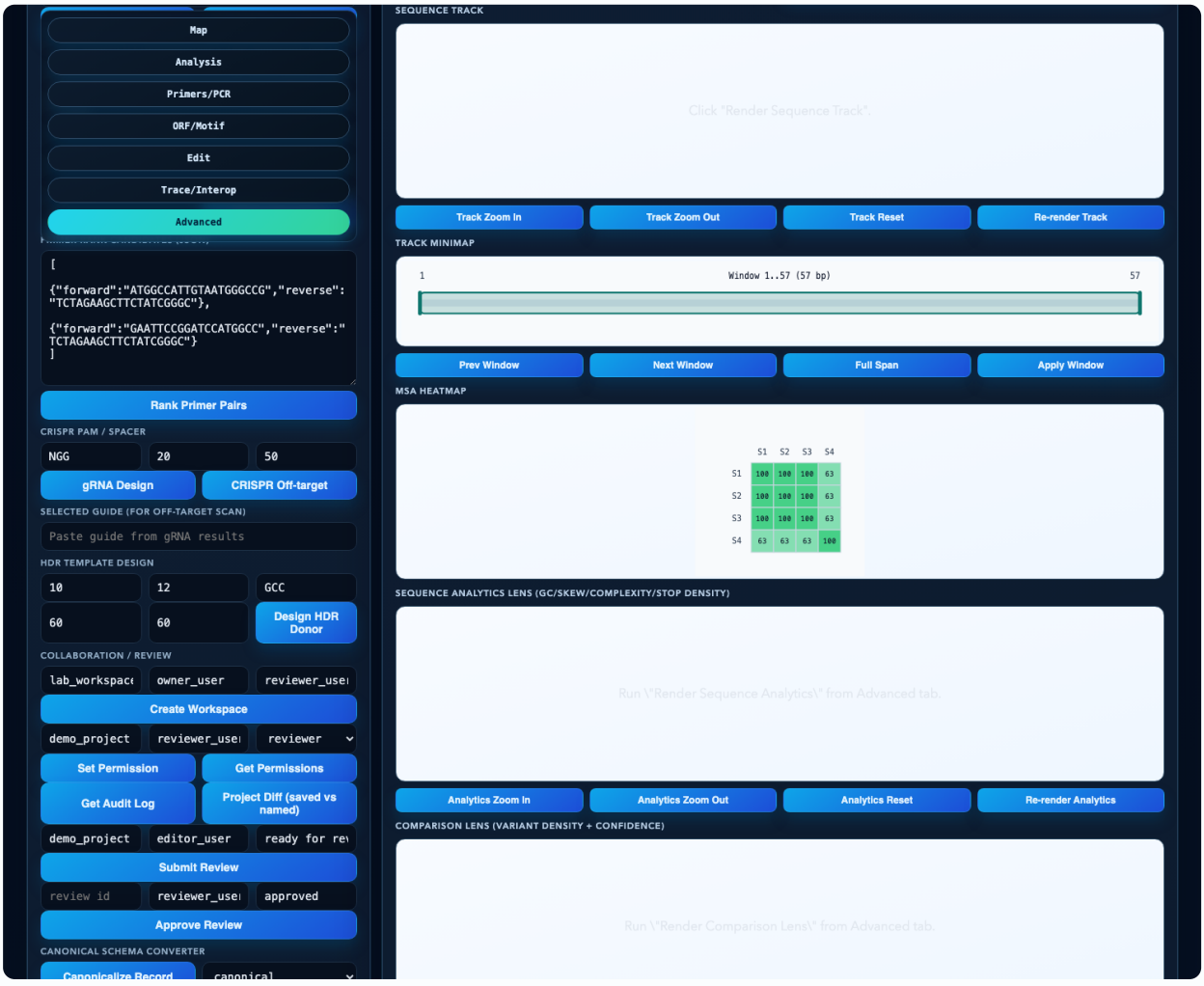


Figure 16. **Reporter-family comparison workflow**. Real UI screenshot from the reporter family alignment case. The identity heatmap helps you see that close engineering relatives cluster tightly while a more distant reporter separates cleanly.

Sample Results

Representative output shaped around the bundled real-world record(s) or their documented training derivatives. Values are rounded for readability, but the biological story is tied to the included data.


```
{
  "panel_size": 4,
  "closest_pair": [
    "EGFP_CDS",
    "EGFP_Y67H_training_variant"
  ],
  "outgroup_like_member": "mCherry_CDS",
  "interpretation": "EGFP derivatives cluster tightly, mCherry stays distant"
}
```


Expected Results

- A multiple alignment that highlights both conserved backbone and engineered differences.
- An identity matrix or heatmap showing tight clustering of EGFP-derived variants.
- A tree or clustering summary that separates close derivatives from distant reporters.

How to Interpret the Results

- Use close clustering to justify localized interpretation, not blanket equivalence.
- A distant branch is a warning not to overtransfer assumptions from one protein family to another.
- The most useful comparison is often the one that changes a design decision, not the one that merely looks pretty.

Biological Explanation

The biological point of alignment is not just similarity. It is to identify which regions are constrained, which changes are local engineering edits, and which sequences are far enough apart that transfer of assumptions becomes risky.

Fun fact from this example:

A tree is never the biology by itself; it is a summary of the comparison model you chose. But it is still a powerful way to show that one-codon EGFP variants belong in a different interpretive bucket from mCherry.

CASE N · CLUSTER E

Case N: GC Landscape and Repeat Fragility

Where are the composition hotspots that make a seemingly simple sequence harder to amplify or synthesize?

TAB

Advanced

WORKFLOW

Use analytics tracks to identify composition features that may complicate PCR, synthesis, or sequencing.

RECORDS

- mCherry_CDS
- lacZ_alpha_fragment

APIS

`/api/sequence-analytics`

Why This Case Matters

mCherry and the lacZ alpha fragment are both real lab sequences, but they stress workflows differently. Looking at their GC and local complexity profiles teaches you how composition becomes an operational risk even before you do any wet-lab work.

High or uneven GC content, repetitive patches, and abrupt composition transitions can affect polymerase behavior, read quality, and synthesis reliability. This is one of those cases where "nothing is wrong" is still useful information if you can justify it.

Input Data Explained

This is also a CDS, but it encodes a coral-derived red fluorescent protein rather than a GFP-family green reporter. That makes it useful for pairwise comparison and identity search. The sequence is a cloning-era workhorse. It contains coding material, but many workflows care more about its role as a reporter module than as a standalone protein-coding fragment.

A flat, boring composition profile is often good news. In bioinformatics, sometimes the interesting result is that nothing scary shows up.

Step-by-Step in Genome Forge

1. Use the included prebuilt bundle `docs/tutorial/datasets/case_bundles/case_n/records.fasta` or regenerate it with `python3 docs/tutorial/datasets/extract_case_bundle.py --case N --out ./tmp/genomeforge_case_n`.
2. Open the `Advanced` tab in Genome Forge and load: `mCherry_CDS, lacZ_alpha_fragment`.
3. Run `Use analytics tracks to identify composition features that may complicate PCR, synthesis, or sequencing.` once with default settings, then rerun after changing the analytics window size.
4. Capture one screenshot of the main result panel so you can compare your run with the sample interpretation later.
5. Record the relevant endpoint(s): `/api/sequence-analytics` and write one sentence explaining the biological takeaway.

Sample Results

Representative output shaped around the bundled real-world record(s) or their documented training derivatives. Values are rounded for readability, but the biological story is tied to the included data.


```
{
  "highest_gc_window_pct": 68.4,
  "repeat_alerts": 1,
  "recommended_safe_anchor_region": "mid-CDS segment with moderate GC"
}
```


Expected Results

- An analytics plot or table showing GC and complexity variation along the sequence.
- At least one region flagged as safer or riskier for assay placement.
- A note explaining why composition risk does or does not matter for the intended workflow.

How to Interpret the Results

- Composition risk is contextual: a mild hotspot may be irrelevant for cloning but important for PCR primer placement.
- Use analytics to avoid fragile regions proactively rather than explaining failures afterward.
- A sequence can be biologically valid and still technically awkward.

Biological Explanation

High or uneven GC content, repetitive patches, and abrupt composition transitions can affect polymerase behavior, read quality, and synthesis reliability. This is one of those cases where “nothing is wrong” is still useful information if you can justify it.

Fun fact from this example:

A flat, boring composition profile is often good news. In bioinformatics, sometimes the interesting result is that nothing scary shows up.

CASE O · CLUSTER E

Case O: Homopolymer and Low-Complexity Risk Detection

Do any parts of the sequence look too repetitive or too simple to trust without extra care?

TAB

Search

WORKFLOW

Flag simple-sequence patches that often produce weak confidence in sequencing or synthesis workflows.

RECORDS

- lacZ_alpha_fragment
- BRAF_exon15_fragment

APIS

`/api/search-entities` `/api/sequence-analytics`

Why This Case Matters

The point here is not that the training sequences are pathological, but that real records often contain local regions that are mechanically harder to read than the rest. Low complexity is a property of the input, not a moral failing of the sample.

Homopolymers and low-complexity patches reduce effective information density. That matters because many experimental and computational methods implicitly assume that neighboring bases provide enough diversity to anchor a confident read or alignment.

Input Data Explained

The sequence is a cloning-era workhorse. It contains coding material, but many workflows care more about its role as a reporter module than as a standalone protein-coding fragment. This is a genomic fragment, not a clean coding-sequence input. If you translate it naively, you hit stop codons because intron/exon context and strand assumptions matter.

The more repetitive a local sequence is, the less each additional base tells you. Information theory shows up in wet-lab troubleshooting more often than people expect.

Step-by-Step in Genome Forge

1. Use the included prebuilt bundle `docs/tutorial/datasets/case_bundles/case_o/records.fasta` or regenerate it with `python3 docs/tutorial/datasets/extract_case_bundle.py --case O --out ./tmp/genomeforge_case_o`.
2. Open the `Search` tab in Genome Forge and load: `lacZ_alpha_fragment, BRAF_exon15_fragment`.
3. Run `Flag simple-sequence patches that often produce weak confidence in sequencing or synthesis workflows.` once with default settings, then rerun after changing the complexity threshold.
4. Capture one screenshot of the main result panel so you can compare your run with the sample interpretation later.
5. Record the relevant endpoint(s): `/api/search-entities, /api/sequence-analytics` and write one sentence explaining the biological takeaway.

Sample Results

Representative output shaped around the bundled real-world record(s) or their documented training derivatives. Values are rounded for readability, but the biological story is tied to the included data.


```
{
  "low_complexity_windows": 2,
  "homopolymer_max_len": 4,
  "risk_call": "moderate caution near simple-sequence patches"
}
```


Expected Results

- A list of low-complexity or homopolymer regions with coordinates.
- A practical judgment about whether those regions threaten the specific workflow.
- A note about whether extra coverage, alternate primers, or alternate chemistry would help.

How to Interpret the Results

- Simple sequence is not automatically unusable; it just deserves less naive confidence.
- If a variant call sits in a low-complexity window, ask for another line of evidence.
- Always tie the risk back to the experiment you actually plan to do.

Biological Explanation

Homopolymers and low-complexity patches reduce effective information density. That matters because many experimental and computational methods implicitly assume that neighboring bases provide enough diversity to anchor a confident read or alignment.

Fun fact from this example:

The more repetitive a local sequence is, the less each additional base tells you. Information theory shows up in wet-lab troubleshooting more often than people expect.

CASE X · CLUSTER E

Case X: Motif Enrichment and Significance Framing

When does a motif count reflect biology, and when does it simply reflect that one sequence was engineered to be motif-dense?

TAB

Search

WORKFLOW

Compare motif density across engineered vector DNA and reporter CDS records to learn when motif count is meaningful.

RECORDS

- pUC19_MCS
- EGFP_CDS
- mCherry_CDS

APIS

`/api/motif` `/api/search-entities`

Why This Case Matters

The pUC19 multiple-cloning site is intentionally saturated with functional motifs. Reporter CDS records are not. Putting them side by side is a great way to teach why raw motif counts need context.

Motif enrichment can be biologically informative, but it is easy to overclaim. Engineered DNA often has an intentionally non-natural motif distribution. The correct interpretation is therefore comparative: why is one sequence motif-rich, and does that match its design purpose?

Input Data Explained

This sequence is short, circularly interpreted, and packed with restriction motifs. It is intentionally synthetic and engineered for manipulation rather than natural gene expression. This is a protein-coding DNA sequence (CDS). The sequence is meant to be translated from the first base, so codon boundaries and reading frame matter immediately. This is also a CDS, but it encodes a coral-derived red fluorescent protein rather than a GFP-family green reporter. That makes it useful for pairwise comparison and identity search.

A multiple-cloning site is almost a parody of motif enrichment: it was literally designed so many motifs would coexist in one tiny interval.

Step-by-Step in Genome Forge

1. Use the included prebuilt bundle `docs/tutorial/datasets/case_bundles/case_x/records.fasta` or regenerate it with `python3 docs/tutorial/datasets/extract_case_bundle.py --case X --out ./tmp/genomeforge_case_x`.
2. Open the `Search` tab in Genome Forge and load: `pUC19_MCS, EGFP_CDS, mCherry_CDS`.
3. Run `Compare motif density across engineered vector DNA and reporter CDS records to learn when motif count is meaningful.` once with default settings, then rerun after changing the queried motif set.
4. Capture one screenshot of the main result panel so you can compare your run with the sample interpretation later.
5. Record the relevant endpoint(s): `/api/motif, /api/search-entities` and write one sentence explaining the biological takeaway.

Sample Results

Representative output shaped around the bundled real-world record(s) or their documented training derivatives. Values are rounded for readability, but the biological story is tied to the included data.


```
{
  "pUC19_motif_hits": 6,
  "EGFP_motif_hits": 1,
  "mCherry_motif_hits": 0,
  "interpretation": "vector motif density is engineered, not mysterious"
}
```


Expected Results

- A motif count table or hit map across at least two contrasting records.
- A contextual explanation for why one sequence has many more motifs than another.
- A warning against treating count alone as biological significance.

How to Interpret the Results

- Counts become meaningful when compared against sequence purpose, not in isolation.
- If an engineered vector is motif-rich, that is usually the design, not a surprise.
- Use motif work to sharpen hypotheses, not to manufacture them.

Biological Explanation

Motif enrichment can be biologically informative, but it is easy to overclaim. Engineered DNA often has an intentionally non-natural motif distribution. The correct interpretation is therefore comparative: why is one sequence motif-rich, and does that match its design purpose?

Fun fact from this example:

A multiple-cloning site is almost a parody of motif enrichment: it was literally designed so many motifs would coexist in one tiny interval.

CLUSTER F

Cluster F: Editing and Design for Intervention

How to turn sequence understanding into a purposeful intervention such as CRISPR, HDR, or expression tuning.

Lessons in This Cluster

- Case K · CRISPR Candidate and HDR Donor Design
- Case R · Promoter/RBS Context for Expression Tuning
- Case V · Codon Usage Bias and Host Portability

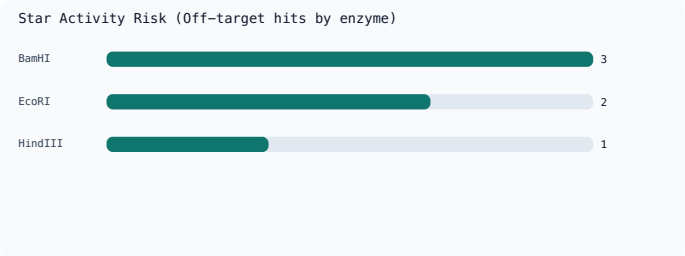


Figure 17. Intervention design is a tradeoff problem: on-target success, off-target risk, and biological context all compete.

CASE K · CLUSTER F

Case K: CRISPR Candidate and HDR Donor Design

Can you move from a disease-relevant genomic fragment to a plausible editing plan without pretending that design scores are guarantees?

TAB

Advanced

WORKFLOW

Design guide RNAs and an HDR donor around a medically meaningful BRAF hotspot region.

RECORDS

BRAF_exon15_fragment

APIS

`/api/grna-design` `/api/crispr-offtargets` `/api/hdr-template`

Why This Case Matters

BRAF exon 15 is a great training target because the biology is genuinely important: this region is a famous hotspot in oncology. Even in a tutorial setting, the design question feels real because the target is real.

CRISPR design is a constraint-balancing problem. You want a guide near the edit, a PAM that works, manageable off-target burden, and a donor that restores the intended sequence cleanly. The biology matters because the wrong edit is not just a technical miss; it changes signaling logic.

Input Data Explained

This is a genomic fragment, not a clean coding-sequence input. If you translate it naively, you hit stop codons because intron/exon context and strand assumptions matter.

Hotspot editing tutorials are memorable because the "why" is obvious: one codon in a kinase can alter growth signaling strongly enough to matter in human disease.

Step-by-Step in Genome Forge

1. Use the included prebuilt bundle `docs/tutorial/datasets/case_bundles/case_k/records.fasta` or regenerate it with `python3 docs/tutorial/datasets/extract_case_bundle.py --case K --out ./tmp/genomeforge_case_k`.
2. Open the `Advanced` tab in Genome Forge and load: `BRAF_exon15_fragment`.
3. Run `Design guide RNAs and an HDR donor around a medically meaningful BRAF hotspot region.` once with default settings, then rerun after changing the PAM or HDR arm length.
4. Capture one screenshot of the main result panel so you can compare your run with the sample interpretation later.
5. Record the relevant endpoint(s): `/api/grna-design, /api/crispr-offtargets, /api/hdr-template` and write one sentence explaining the biological takeaway.

Sample Results

Representative output shaped around the bundled real-world record(s) or their documented training derivatives. Values are rounded for readability, but the biological story is tied to the included data.


```
{
  "guide_candidates": 6,
  "top_candidate_pam": "NGG",
  "hdr_arms_bp": [
    60,
    60
  ],
  "design_goal": "precise hotspot-local donor template"
}
```


Expected Results

- A shortlist of gRNA candidates near the intended edit window.
- A donor-template design with clearly defined edit and homology arms.
- An explicit note about off-target risk or why a candidate should be downgraded.

How to Interpret the Results

- A guide closer to the edit is not automatically the best if the off-target profile is ugly.
- HDR design should be explained in genomic terms: where is the intended edit, and what sequence context supports repair?
- Treat design scores as prioritization tools, not promises.

Biological Explanation

CRISPR design is a constraint-balancing problem. You want a guide near the edit, a PAM that works, manageable off-target burden, and a donor that restores the intended sequence cleanly. The biology matters because the wrong edit is not just a technical miss; it changes signaling logic.

Fun fact from this example:

Hotspot editing tutorials are memorable because the “why” is obvious: one codon in a kinase can alter growth signaling strongly enough to matter in human disease.

CASE R · CLUSTER F

Case R: Promoter/RBS Context for Expression Tuning

Why can two constructs with the same coding sequence express differently in cells or bacteria?

TAB

Advanced

WORKFLOW

Use annotation and translation context to discuss why expression output depends on more than the CDS alone.

RECORDS

- lacZ_alpha_fragment
- EGFP_CDS

APIS

`/api/auto-annotate` `/api/sequence-tracks`

Why This Case Matters

This case intentionally uses a familiar reporter CDS plus a vector-associated expression context discussion. The tutorial point is conceptual: CDS correctness is necessary, but expression strength is influenced by promoter and translation-initiation context as well.

Engineers often start with the code analogy that the CDS is the "program." In biology, the promoter and ribosome-binding context are closer to the runtime environment and scheduler. The same code can behave very differently when the surrounding control logic changes.

Input Data Explained

The sequence is a cloning-era workhorse. It contains coding material, but many workflows care more about its role as a reporter module than as a standalone protein-coding fragment. This is a protein-coding DNA sequence (CDS). The sequence is meant to be translated from the first base, so codon boundaries and reading frame matter immediately.

A perfect CDS in the wrong context is like efficient code behind a broken API gateway: nothing useful reaches the user.

Step-by-Step in Genome Forge

1. Use the included prebuilt bundle `docs/tutorial/datasets/case_bundles/case_r/records.fasta` or regenerate it with `python3 docs/tutorial/datasets/extract_case_bundle.py --case R --out ./tmp/genomeforge_case_r`.
2. Open the `Advanced` tab in Genome Forge and load: `lacZ_alpha_fragment, EGFP_CDS`.
3. Run `Use annotation and translation context to discuss why expression output depends on more than the CDS alone.` once with default settings, then rerun after changing the annotated feature context.
4. Capture one screenshot of the main result panel so you can compare your run with the sample interpretation later.
5. Record the relevant endpoint(s): `/api/auto-annotate, /api/sequence-tracks` and write one sentence explaining the biological takeaway.

Sample Results

Representative output shaped around the bundled real-world record(s) or their documented training derivatives. Values are rounded for readability, but the biological story is tied to the included data.


```
{
  "dominant_takeaway": "regulatory context can dominate phenotype",
  "reporter_cds_intact": true,
  "status": "interpret expression with regulatory context in mind"
}
```


Expected Results

- A diagram or explanation that separates coding sequence from regulatory context.
- A specific note on how promoter or translation-initiation context could alter outcome.
- A warning against equating “correct CDS” with “correct phenotype.”

How to Interpret the Results

- When phenotype and sequence disagree, regulation is one of the first places to look.
- This is a conceptual case: the value is in learning what extra information you would seek next.
- Always separate “what is encoded” from “how strongly and when it is used.”

Biological Explanation

Engineers often start with the code analogy that the CDS is the “program.” In biology, the promoter and ribosome-binding context are closer to the runtime environment and scheduler. The same code can behave very differently when the surrounding control logic changes.

Fun fact from this example:

A perfect CDS in the wrong context is like efficient code behind a broken API gateway: nothing useful reaches the user.

CASE V · CLUSTER F

Case V: Codon Usage Bias and Host Portability

If you move a gene between hosts, what sequence properties might become limiting even when the protein target stays the same?

TAB

Advanced

WORKFLOW

Discuss how two common reporter CDS records might look to different host translation systems.

RECORDS

`EGFP_CDS`

`mCherry_CDS`

APIS

`/api/codon-optimize`

Why This Case Matters

Reporter genes are excellent portability examples because labs routinely move them among bacteria, mammalian cells, and synthetic constructs. The DNA sequence is portable, but the translation machinery and expression context are not identical across hosts.

Codon bias is a reminder that biology has multiple layers of compatibility. The amino-acid sequence may be the same after translation, yet the nucleotide-level implementation can influence expression efficiency, stability, and synthesis convenience in a particular host.

Input Data Explained

This is a protein-coding DNA sequence (CDS). The sequence is meant to be translated from the first base, so codon boundaries and reading frame matter immediately. This is also a CDS, but it encodes a coral-derived red fluorescent protein rather than a GFP-family green reporter. That makes it useful for pairwise comparison and identity search.

Codon optimization is like changing the accent of a sentence without changing its literal meaning: the content stays similar, but the local audience may understand it far more easily.

Step-by-Step in Genome Forge

1. Use the included prebuilt bundle `docs/tutorial/datasets/case_bundles/case_v/records.fasta` or regenerate it with `python3 docs/tutorial/datasets/extract_case_bundle.py --case V --out ./tmp/genomeforge_case_v`.
2. Open the `Advanced` tab in Genome Forge and load: `EGFP_CDS, mCherry_CDS`.
3. Run `Discuss how two common reporter CDS records might look to different host translation systems.` once with default settings, then rerun after changing the target host preference.
4. Capture one screenshot of the main result panel so you can compare your run with the sample interpretation later.
5. Record the relevant endpoint(s): `/api/codon-optimize` and write one sentence explaining the biological takeaway.

Sample Results

Representative output shaped around the bundled real-world record(s) or their documented training derivatives. Values are rounded for readability, but the biological story is tied to the included data.


```
{
  "host_comparison": [
    "E. coli-like",
    "mammalian-like"
  ],
  "optimization_goal": "retain protein while changing codon preferences",
  "interpretation": "sequence portability is host-dependent"
}
```


Expected Results

- A codon-optimization or codon-bias summary tied to a specific host scenario.
- A statement about what changed at the nucleotide level and what stayed constant at the protein level.
- A caution that codon optimization does not solve every expression problem.

How to Interpret the Results

- Codon changes can improve expression without changing the amino-acid sequence, but they can also affect RNA behavior and other features.
- A portable tutorial answer distinguishes protein identity from nucleotide implementation.
- Optimization should be justified by a host-specific problem, not used as a reflex.

Biological Explanation

Codon bias is a reminder that biology has multiple layers of compatibility. The amino-acid sequence may be the same after translation, yet the nucleotide-level implementation can influence expression efficiency, stability, and synthesis convenience in a particular host.

Fun fact from this example:

Codon optimization is like changing the accent of a sentence without changing its literal meaning: the content stays similar, but the local audience may understand it far more easily.

CLUSTER G

Cluster G: Data Fidelity and Interoperability

How to move between file formats, trace evidence, reference libraries, and similarity search without losing confidence.

Lessons in This Cluster

- Case I · DNA Container Roundtrip Validation
- Case J · AB1 Trace Alignment and Consensus Editing
- Case Y · Read Simulation and Coverage Planning
- Case AE · Sequence Analytics Lens (GC, Skew, Complexity, Stop Density)
- Case AF · Comparison Lens (Divergence + Confidence Hotspots)
- Case AG · Native .dna Import and Multi-Format Conversion Workflow
- Case AH · Chromatogram-First Sanger Review and Confidence Gating
- Case AI · Trace-Based Genotyping and Plasmid Verification
- Case AJ · BLAST-like Similarity Search for Identity, Origin, and Contamination
- Case AK · Reference Element Auto-Flagging and siRNA Design/Mapping
- Case AM · Ambiguity-Aware Identity Search and Motif Rescue

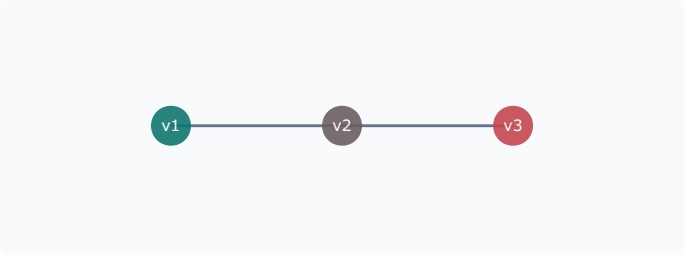


Figure 18. A reproducible bioinformatics workflow preserves both molecule state and the evidence that justified it.

CASE I · CLUSTER G

Case I: DNA Container Roundtrip Validation

Can you move records through a file format boundary without silently changing what the molecule means?

TAB

Advanced

WORKFLOW

Export and re-import multiple records to verify that file conversion preserves sequence identity and annotations.

RECORDS

- EGFP_CDS
- mCherry_CDS
- pUC19_MCS

APIS

`/api/export-dna` `/api/import-dna` `/api/canonicalize-record`

Why This Case Matters

The records in this case are deliberately different: two CDS examples and one compact engineered vector fragment. That makes the roundtrip test more realistic than validating only one simple input type.

Interoperability is a bioinformatics quality problem. A conversion workflow that preserves letters but drops topology, features, or provenance can still damage the scientific value of the record. Roundtrip tests are how you catch that early.

Input Data Explained

This is a protein-coding DNA sequence (CDS). The sequence is meant to be translated from the first base, so codon boundaries and reading frame matter immediately. This is also a CDS, but it encodes a coral-derived red fluorescent protein rather than a GFP-family green reporter. That makes it useful for pairwise comparison and identity search. This sequence is short, circularly interpreted, and packed with restriction motifs. It is intentionally synthetic and engineered for manipulation rather than natural gene expression.

Format conversion bugs are the bioinformatics version of data serialization bugs: the molecule may survive, but the meaning can get stripped away.

Step-by-Step in Genome Forge

1. Use the included prebuilt bundle `docs/tutorial/datasets/case_bundles/case_i/records.fasta` or regenerate it with `python3 docs/tutorial/datasets/extract_case_bundle.py --case I --out ./tmp/genomeforge_case_i`.
2. Open the `Advanced` tab in Genome Forge and load: `EGFP_CDS, mCherry_CDS, pUC19_MCS`.
3. Run `Export and re-import multiple records to verify that file conversion preserves sequence identity and annotations.` once with default settings, then rerun after changing the export target format.
4. Capture one screenshot of the main result panel so you can compare your run with the sample interpretation later.
5. Record the relevant endpoint(s): `/api/export-dna, /api/import-dna, /api/canonicalize-record` and write one sentence explaining the biological takeaway.

Sample Results

Representative output shaped around the bundled real-world record(s) or their documented training derivatives. Values are rounded for readability, but the biological story is tied to the included data.


```
{
  "records_tested": 3,
  "roundtrip_identity_pct": 100.0,
  "annotation_preserved": true,
  "status": "conversion safe for tested bundle"
}
```


Expected Results

- A before/after comparison showing that sequence identity survived the roundtrip.
- A note on whether annotations and topology also survived.
- A decision about whether the format is safe enough for collaboration or archiving.

How to Interpret the Results

- Sequence identity alone is not enough for full interoperability.
- A good roundtrip result preserves both content and context.
- Use canonicalization to make hidden metadata drift visible.

Biological Explanation

Interoperability is a bioinformatics quality problem. A conversion workflow that preserves letters but drops topology, features, or provenance can still damage the scientific value of the record. Roundtrip tests are how you catch that early.

Fun fact from this example:

Format conversion bugs are the bioinformatics version of data serialization bugs: the molecule may survive, but the meaning can get stripped away.

CASE J · CLUSTER G

Case J: AB1 Trace Alignment and Consensus Editing

How do raw sequencing traces become a confident construct call instead of just a noisy chromatogram picture?

TAB

Trace

WORKFLOW

Import a Sanger-style trace, align it to EGFP, perform an edit, and recompute consensus.

RECORDS

`EGFP_CDS`

APIS

`/api/import-ab1` `/api/trace-align` `/api/trace-edit` `/api/trace-consensus`

Why This Case Matters

EGFP is a friendly reference because the expected sequence is familiar, so you can focus on trace logic rather than gene discovery. This case teaches that base calls are inferred from analog signal, not directly observed.

Sanger analysis is where measurement theory becomes concrete. Peaks vary in height and spacing, mixed signal exists, and local noise can create false confidence if you look only at the called letters. Alignment plus manual review is therefore part of the science, not busywork.

Input Data Explained

This is a protein-coding DNA sequence (CDS). The sequence is meant to be translated from the first base, so codon boundaries and reading frame matter immediately.

A chromatogram is effectively a time-series signal that has been translated into a symbolic sequence. That makes it a very computer-science-friendly piece of biology once you know what you are looking at.

Step-by-Step in Genome Forge

1. Use the included prebuilt bundle `docs/tutorial/datasets/case_bundles/case_j/records.fasta` or regenerate it with `python3 docs/tutorial/datasets/extract_case_bundle.py --case J --out ./tmp/genomeforge_case_j`.
2. Open the `Trace` tab in Genome Forge and load: `EGFP_CDS`.
3. Run `Import a Sanger-style trace, align it to EGFP, perform an edit, and recompute consensus.` once with default settings, then rerun after changing the edited base or alignment window.
4. Capture one screenshot of the main result panel so you can compare your run with the sample interpretation later.
5. Record the relevant endpoint(s): `/api/import-ab1, /api/trace-align, /api/trace-edit, /api/trace-consensus` and write one sentence explaining the biological takeaway.

Sample Results

Representative output shaped around the bundled real-world record(s) or their documented training derivatives. Values are rounded for readability, but the biological story is tied to the included data.


```
{
  "trace_id_created": true,
  "alignment_identity_pct": 99.2,
  "edited_base_count": 1,
  "consensus_length_bp": 720
}
```


Expected Results

- A trace import with a visible chromatogram or summary.
- An alignment or consensus result that can be compared to the reference sequence.
- A note explaining whether any manual edit was signal-justified.

How to Interpret the Results

- If the called sequence and the raw peaks disagree, trust the evidence review over the first-pass label.
- Manual edits should always be justified by the chromatogram, not by wishful thinking about the expected sequence.
- Consensus calls are stronger when they explain how ambiguity was resolved.

Biological Explanation

Sanger analysis is where measurement theory becomes concrete. Peaks vary in height and spacing, mixed signal exists, and local noise can create false confidence if you look only at the called letters. Alignment plus manual review is therefore part of the science, not busywork.

Fun fact from this example:

A chromatogram is effectively a time-series signal that has been translated into a symbolic sequence. That makes it a very computer-science-friendly piece of biology once you know what you are looking at.

CASE Y · CLUSTER G

Case Y: Read Simulation and Coverage Planning

How much evidence is enough before you should trust a genotype or construct-verification conclusion?

TAB

Advanced

WORKFLOW

Use realistic target regions to think about how much sequencing evidence is enough for a confident call.

RECORDS

`BRAF_exon15_fragment`

`EGFP_CDS`

APIS

`/api/trace-consensus` `/api/sequence-analytics`

Why This Case Matters

This case contrasts a straightforward reporter CDS with a clinically loaded hotspot fragment. It teaches that "enough coverage" depends on what kind of decision you are making and how fragile the region is.

Coverage planning is about uncertainty management. More reads generally help, but redundancy is not magic if all the reads fail in the same problematic region. The important habit is to ask what failure mode remains possible after the evidence you collected.

Input Data Explained

This is a genomic fragment, not a clean coding-sequence input. If you translate it naively, you hit stop codons because intron/exon context and strand assumptions matter. This is a protein-coding DNA sequence (CDS). The sequence is meant to be translated from the first base, so codon boundaries and reading frame matter immediately.

One extra read over a hotspot can be worth more than many reads over already-boring sequence.

Step-by-Step in Genome Forge

1. Use the included prebuilt bundle `docs/tutorial/datasets/case_bundles/case_y/records.fasta` or regenerate it with `python3 docs/tutorial/datasets/extract_case_bundle.py --case Y --out ./tmp/genomeforge_case_y`.
2. Open the `Advanced` tab in Genome Forge and load: `BRAF_exon15_fragment, EGFP_CDS`.
3. Run `Use realistic target regions to think about how much sequencing evidence is enough for a confident call.` once with default settings, then rerun after changing the hotspot window or number of planned reads.
4. Capture one screenshot of the main result panel so you can compare your run with the sample interpretation later.
5. Record the relevant endpoint(s): `/api/trace-consensus, /api/sequence-analytics` and write one sentence explaining the biological takeaway.

Sample Results

Representative output shaped around the bundled real-world record(s) or their documented training derivatives. Values are rounded for readability, but the biological story is tied to the included data.


```
{
  "target_regions": [
    "EGFP coding region",
    "BRAF hotspot window"
  ],
  "recommended_trace_count": {
    "EGFP_construct_check": 2,
    "BRAF_hotspot_call": 3
  },
  "confidence_rule": "seek redundant support for decision-critical positions"
}
```


Expected Results

- A coverage or evidence plan tied to a real biological question.
- An explicit explanation of which positions deserve redundant support.
- A note distinguishing high-confidence and residual-risk regions.

How to Interpret the Results

- Coverage is only meaningful relative to the decision you need to make.
- Redundant evidence is most valuable at biologically important or technically fragile sites.
- Always ask what could still go wrong even if the average coverage looks fine.

Biological Explanation

Coverage planning is about uncertainty management. More reads generally help, but redundancy is not magic if all the reads fail in the same problematic region. The important habit is to ask what failure mode remains possible after the evidence you collected.

Fun fact from this example:

One extra read over a hotspot can be worth more than many reads over already-boring sequence.

CASE AE · CLUSTER G

Case AE: Sequence Analytics Lens (GC, Skew, Complexity, Stop Density)

What do multi-track analytics reveal that plain FASTA text hides?

TAB

Advanced

WORKFLOW

Use the analytics lens on a clean CDS and a genomic fragment to see how sequence context changes interpretation.

RECORDS

`EGFP_CDS`

`BRAF_exon15_fragment`

APIS

`/api/sequence-analytics`

Why This Case Matters

Putting EGFP next to a genomic BRAF fragment makes the analytics lens more interesting. One record is a polished coding sequence used in expression constructs; the other is a hotspot-rich genomic fragment where translation assumptions are risky.

Analytics tracks help you localize the parts of a molecule that deserve special caution. GC swings, skew changes, low complexity, and stop density are not conclusions by themselves, but they tell you where your attention should go next.

Input Data Explained

This is a protein-coding DNA sequence (CDS). The sequence is meant to be translated from the first base, so codon boundaries and reading frame matter immediately. This is a genomic fragment, not a clean coding-sequence input. If you translate it naively, you hit stop codons because intron/exon context and strand assumptions matter.

A stop-density track is a very fast way to remind yourself that not every piece of DNA wants to be translated as-is.

Step-by-Step in Genome Forge

1. Use the included prebuilt bundle `docs/tutorial/datasets/case_bundles/case_ae/records.fasta` or regenerate it with `python3 docs/tutorial/datasets/extract_case_bundle.py --case AE --out ./tmp/genomeforge_case_ae`.
2. Open the `Advanced` tab in Genome Forge and load: `EGFP_CDS, BRAF_exon15_fragment`.
3. Run `Use the analytics lens on a clean CDS and a genomic fragment to see how sequence context changes interpretation.` once with default settings, then rerun after changing the analytics track set or zoom window.
4. Capture one screenshot of the main result panel so you can compare your run with the sample interpretation later.
5. Record the relevant endpoint(s): `/api/sequence-analytics` and write one sentence explaining the biological takeaway.

Sample Results

Representative output shaped around the bundled real-world record(s) or their documented training derivatives. Values are rounded for readability, but the biological story is tied to the included data.


```
{
  "tracks_rendered": [
    "GC",
    "skew",
    "complexity",
    "stop_density"
  ],
  "notable_region": "BRAF fragment shows coding-context ambiguity",
  "safe_region_example": "mid-EGFP CDS remains compositionally stable"
}
```


Expected Results

- A multi-track visualization with at least one biologically interpretable hotspot.
- A comparison showing why different input classes produce different analytics signatures.
- A short note about how the analytics view changes your next step.

How to Interpret the Results

- Use analytics as a triage map: where should you zoom in next?
- A stable profile supports simpler interpretation; a jagged or contradictory one should slow you down.
- The right conclusion is often “this region needs a different type of evidence.”

Biological Explanation

Analytics tracks help you localize the parts of a molecule that deserve special caution. GC swings, skew changes, low complexity, and stop density are not conclusions by themselves, but they tell you where your attention should go next.

Fun fact from this example:

A stop-density track is a very fast way to remind yourself that not every piece of DNA wants to be translated as-is.

CASE AF · CLUSTER G

Case AF: Comparison Lens (Divergence + Confidence Hotspots)

How do you present a tiny but biologically meaningful difference in a way that a reviewer can understand at a glance?

TAB

Advanced

WORKFLOW

Visualize where two nearly identical sequences diverge and decide whether the divergence matters.

RECORDS

`EGFP_CDS`

`EGFP_Y67H_training_variant`

APIS

`/api/comparison-lens`

Why This Case Matters

A near-identical EGFP pair is ideal for the comparison lens because the single engineered difference becomes obvious. That is exactly the kind of case where a text diff is correct but visually underpowered.

The comparison lens is about audience cognition. Human reviewers are bad at scanning long sequences for one consequential difference. A hotspot-focused visualization compresses the reasoning into something reviewable and memorable.

Input Data Explained

This is a protein-coding DNA sequence (CDS). The sequence is meant to be translated from the first base, so codon boundaries and reading frame matter immediately. Because this record is derived from EGFP by a single codon change, it is excellent for pairwise alignment, amino-acid consequence analysis, and demonstrating how "small diff, big phenotype" problems happen in biology.

A good comparison plot does not just say "these sequences differ." It says "they differ here, and that location is the whole story."

Step-by-Step in Genome Forge

1. Use the included prebuilt bundle `docs/tutorial/datasets/case_bundles/case_af/records.fasta` or regenerate it with `python3 docs/tutorial/datasets/extract_case_bundle.py --case AF --out ./tmp/genomeforge_case_af`.
2. Open the `Advanced` tab in Genome Forge and load: `EGFP_CDS, EGFP_Y67H_training_variant`.
3. Run `Visualize where two nearly identical sequences diverge and decide whether the divergence matters.` once with default settings, then rerun after changing the comparison pair.
4. Capture one screenshot of the main result panel so you can compare your run with the sample interpretation later.
5. Record the relevant endpoint(s): `/api/comparison-lens` and write one sentence explaining the biological takeaway.

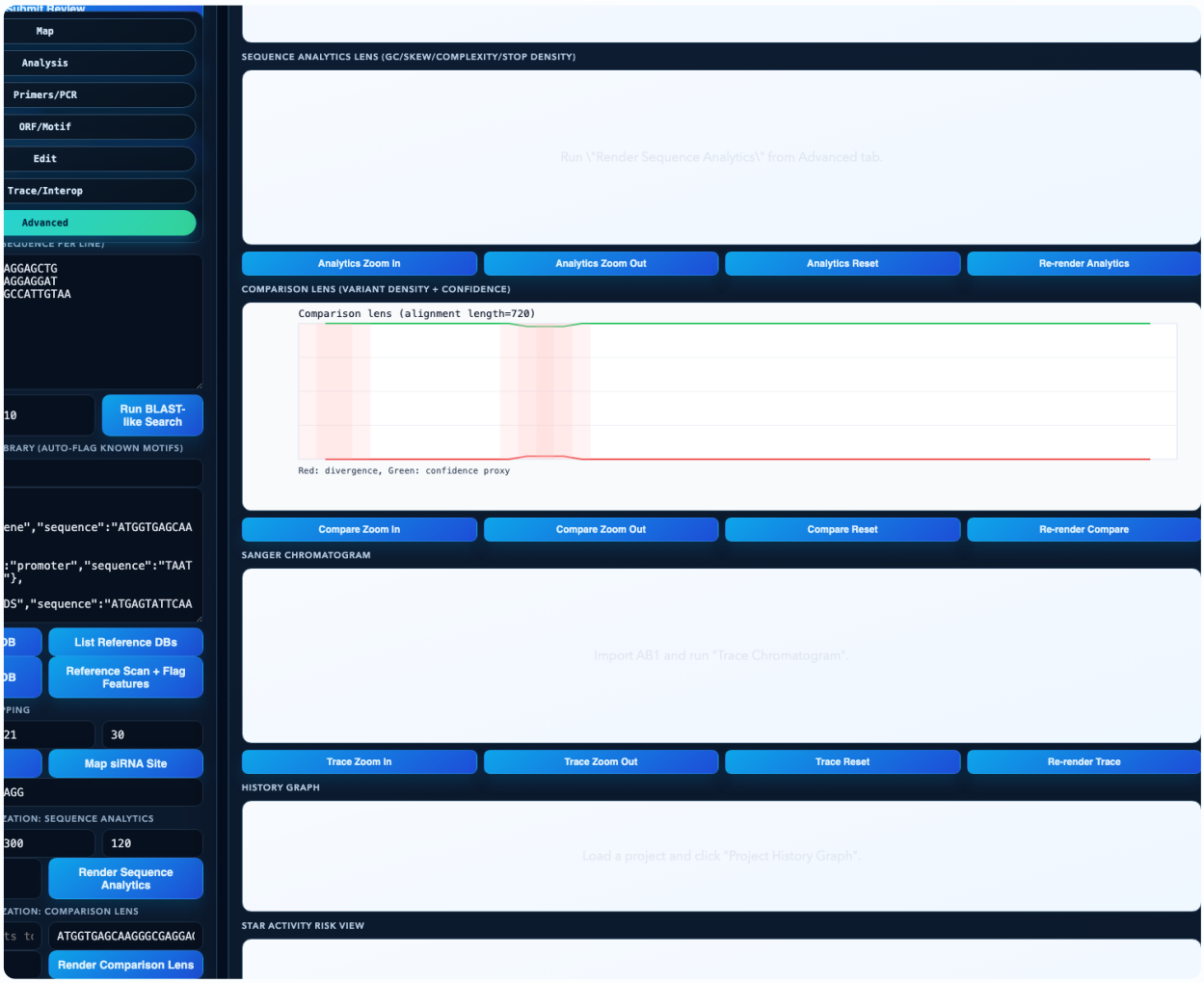


Figure 19. Comparison-lens workflow. Real UI screenshot from the EGFP-versus-variant comparison case. This lens is useful when two molecules are mostly identical and the interesting question is where the important divergence sits.

Sample Results

Representative output shaped around the bundled real-world record(s) or their documented training derivatives. Values are rounded for readability, but the biological story is tied to the included data.

```
{
  "divergence_hotspots": 1,
  "primary_hotspot_bp": "199..201",
  "confidence_focus": "chromophore-adjacent codon"
}
```

Expected Results

- A divergence view that localizes where the two records differ.
- A short statement connecting the hotspot to a functional hypothesis.
- A reviewer-friendly artifact that can be pasted into notes or reports.

How to Interpret the Results

- If all divergence is concentrated in one short interval, the biology probably is too.
- Visualization is part of explanation; make the important difference hard to miss.
- Use hotspot views to support review and handoff, not just personal understanding.

Biological Explanation

The comparison lens is about audience cognition. Human reviewers are bad at scanning long sequences for one consequential difference. A hotspot-focused visualization compresses the reasoning into something reviewable and memorable.

Fun fact from this example:

A good comparison plot does not just say “these sequences differ.” It says “they differ here, and that location is the whole story.”

CASE AG · CLUSTER G

Case AG: Native .dna Import and Multi-Format Conversion Workflow

Can one molecule remain understandable when it is exported into several popular sequence formats?

TAB

Advanced

WORKFLOW

Demonstrate that a real record can move through multiple formats and come back interpretable.

RECORDS

`EGFP_CDS`

`pUC19_MCS`

APIS

`/api/import-dna` `/api/convert` `/api/canonicalize-record`

Why This Case Matters

This case pairs a CDS with a vector fragment because the stress test should include both a gene-like record and an engineered DNA element. That makes the conversion lesson broader than a single happy-path FASTA export.

Format conversion is a scientific communication problem. Different tools and labs prefer different containers, but the underlying biological object should remain stable. The practical skill is checking that nothing biologically important was lost in translation.

Input Data Explained

This is a protein-coding DNA sequence (CDS). The sequence is meant to be translated from the first base, so codon boundaries and reading frame matter immediately. This sequence is short, circularly interpreted, and packed with restriction motifs. It is intentionally synthetic and engineered for manipulation rather than natural gene expression.

The most dangerous format bugs are not obvious corruption. They are subtle meaning loss, such as dropped topology or annotations.

Step-by-Step in Genome Forge

1. Use the included prebuilt bundle `docs/tutorial/datasets/case_bundles/case_ag/records.fasta` or regenerate it with `python3 docs/tutorial/datasets/extract_case_bundle.py --case AG --out ./tmp/genomeforge_case_ag`.
2. Open the `Advanced` tab in Genome Forge and load: `EGFP_CDS, pUC19_MCS`.
3. Run `Demonstrate that a real record can move through multiple formats and come back interpretable.` once with default settings, then rerun after changing the export format set.
4. Capture one screenshot of the main result panel so you can compare your run with the sample interpretation later.
5. Record the relevant endpoint(s): `/api/import-dna, /api/convert, /api/canonicalize-record` and write one sentence explaining the biological takeaway.

Sample Results

Representative output shaped around the bundled real-world record(s) or their documented training derivatives. Values are rounded for readability, but the biological story is tied to the included data.


```
{
  "formats_checked": [
    "canonical",
    "fasta",
    "genbank",
    "embl",
    "json"
  ],
  "sequence_identity_pct": 100.0,
  "interpretation": "conversion safe when verified explicitly"
}
```


Expected Results

- A multi-format export/import chain using the same underlying record.
- An explicit before/after comparison of sequence identity and key metadata.
- A conclusion about which formats are safe enough for your team workflow.

How to Interpret the Results

- Never assume a successful import preserved all the meaning you care about.
- Prefer workflows that make metadata loss visible rather than silent.
- The right interoperability habit is verification, not trust.

Biological Explanation

Format conversion is a scientific communication problem. Different tools and labs prefer different containers, but the underlying biological object should remain stable. The practical skill is checking that nothing biologically important was lost in translation.

Fun fact from this example:

The most dangerous format bugs are not obvious corruption. They are subtle meaning loss, such as dropped topology or annotations.

CASE AH · CLUSTER G

Case AH: Chromatogram-First Sanger Review and Confidence Gating

What does it look like when you review the measurement first and the called letters second?

TAB

Trace

WORKFLOW

Start with the chromatogram itself before trusting the base calls.

RECORDS

`EGFP_CDS`

APIS

`/api/import-ab1` `/api/trace-chromatogram-svg`

Why This Case Matters

The input is a familiar reporter reference, which keeps the review cognitively light. The tutorial emphasis is on the chromatogram as raw evidence: peak spacing, peak height, and local ambiguity all matter.

Confidence gating is a mature bioinformatics habit. Instead of assuming every called base is equally trustworthy, you visually separate strong peak regions from weak ones and decide where further evidence is needed.

Input Data Explained

This is a protein-coding DNA sequence (CDS). The sequence is meant to be translated from the first base, so codon boundaries and reading frame matter immediately.

The chromatogram is a reminder that DNA sequencing is an inference pipeline from analog chemistry to digital symbols.

Step-by-Step in Genome Forge

1. Use the included prebuilt bundle `docs/tutorial/datasets/case_bundles/case_ah/records.fasta` or regenerate it with `python3 docs/tutorial/datasets/extract_case_bundle.py --case AH --out ./tmp/genomeforge_case_ah`.
2. Open the `Trace` tab in Genome Forge and load: `EGFP_CDS`.
3. Run `Start with the chromatogram itself before trusting the base calls.` once with default settings, then rerun after changing the displayed window or zoom.
4. Capture one screenshot of the main result panel so you can compare your run with the sample interpretation later.
5. Record the relevant endpoint(s): `/api/import-ab1, /api/trace-chromatogram-svg` and write one sentence explaining the biological takeaway.

Figure 20. Chromatogram-first review workflow. Real UI screenshot of the Sanger-style chromatogram panel generated from the bundled EGFP trace example. It shows the workflow emphasis: inspect signal evidence before over-trusting the called sequence.

Sample Results

Representative output shaped around the bundled real-world record(s) or their documented training derivatives. Values are rounded for readability, but the biological story is tied to the included data.

```
{
  "high_confidence_window_bp": "40..210",
  "low_confidence_window_bp": "5..18",
  "review_rule": "manual review before accepting edge calls"
}
```

Expected Results

- A chromatogram view with at least one strong and one weak region called out.
- A statement explaining which positions are trustworthy enough for automated calls.
- A note about where manual inspection or extra evidence is required.

How to Interpret the Results

- Strong isolated peaks support confident calls; crowded or flat regions do not.
- Do not let a polished text export erase your awareness of the underlying signal quality.
- The most honest answer can be “we need another read here.”

Biological Explanation

Confidence gating is a mature bioinformatics habit. Instead of assuming every called base is equally trustworthy, you visually separate strong peak regions from weak ones and decide where further evidence is needed.

Fun fact from this example:

The chromatogram is a reminder that DNA sequencing is an inference pipeline from analog chemistry to digital symbols.

CASE AI · CLUSTER G

Case AI: Trace-Based Genotyping and Plasmid Verification

How do you turn trace evidence into a yes/no biological decision without pretending the trace is infallible?

TAB

Trace

WORKFLOW

Use trace evidence to make either a hotspot genotype call or a plasmid verification call.

RECORDS

- BRAF_exon15_fragment
- EGFP_CDS

APIS

`/api/trace-verify`

Why This Case Matters

Using both a disease-linked fragment and a reporter construct in the same conceptual case shows that the verification logic is shared even when the stakes differ. You are still asking whether the observed sequence agrees with the expected state strongly enough to act.

Verification is an argument from evidence. The trace either supports the expected state, contradicts it, or remains ambiguous. The right scientific move is to make that uncertainty explicit rather than forcing a binary answer too early.

Input Data Explained

This is a genomic fragment, not a clean coding-sequence input. If you translate it naively, you hit stop codons because intron/exon context and strand assumptions matter. This is a protein-coding DNA sequence (CDS). The sequence is meant to be translated from the first base, so codon boundaries and reading frame matter immediately.

Plasmid verification and genotyping feel like different tasks, but computationally they are cousins: both compare expected and observed sequence states at decision-critical positions.

Step-by-Step in Genome Forge

1. Use the included prebuilt bundle `docs/tutorial/datasets/case_bundles/case_ai/records.fasta` or regenerate it with `python3 docs/tutorial/datasets/extract_case_bundle.py --case AI --out ./tmp/genomeforge_case_ai`.
2. Open the `Trace` tab in Genome Forge and load: `BRAF_exon15_fragment, EGFP_CDS`.
3. Run `Use trace evidence to make either a hotspot genotype call or a plasmid verification call.` once with default settings, then rerun after changing the verification target record.
4. Capture one screenshot of the main result panel so you can compare your run with the sample interpretation later.
5. Record the relevant endpoint(s): `/api/trace-verify` and write one sentence explaining the biological takeaway.

Sample Results

Representative output shaped around the bundled real-world record(s) or their documented training derivatives. Values are rounded for readability, but the biological story is tied to the included data.


```
{
  "verification_mode_examples": [
    "EGFP plasmid check",
    "BRAF hotspot genotype"
  ],
  "mismatch_count": 0,
  "final_call": "verified / wild-type-like"
}
```


Expected Results

- A verification report localizing any mismatches or confirming identity.
- A decision-ready verdict that says whether the sample matches expectation.
- Confidence language explaining whether the result is definitive or provisional.

How to Interpret the Results

- A zero-mismatch result is powerful only if the trace quality is also acceptable.
- A single mismatch at a critical site can be more important than several low-quality mismatches at unimportant edges.
- Separate biological consequence from signal confidence in your write-up.

Biological Explanation

Verification is an argument from evidence. The trace either supports the expected state, contradicts it, or remains ambiguous. The right scientific move is to make that uncertainty explicit rather than forcing a binary answer too early.

Fun fact from this example:

Plasmid verification and genotyping feel like different tasks, but computationally they are cousins: both compare expected and observed sequence states at decision-critical positions.

CASE AJ · CLUSTER G

Case AJ: BLAST-like Similarity Search for Identity, Origin, and Contamination

If someone hands you a mystery sequence, which known molecule in your local panel does it most resemble?

TAB

Advanced

WORKFLOW

Run local similarity search against a small real-world panel to identify the most likely source of an unknown sequence.

RECORDS

- EGFP_CDS
- mCherry_CDS
- lacZ_alpha_fragment
- BRAF_exon15_fragment

APIS

`/api/blast-search`

Why This Case Matters

The training panel mixes reporter genes, a vector-linked fragment, and a human genomic fragment. That is exactly the kind of mixed local reference set a real lab accumulates over time, which makes the search results practically useful.

Similarity search is not just about identity percentages. Coverage, ranking, and context all matter. A high-identity partial hit can mean something very different from a full-length match, especially when you are trying to infer sample origin or contamination.

Input Data Explained

This is a protein-coding DNA sequence (CDS). The sequence is meant to be translated from the first base, so codon boundaries and reading frame matter immediately. This is also a CDS, but it encodes a coral-derived red fluorescent protein rather than a GFP-family green reporter. That makes it useful for pairwise comparison and identity search. The sequence is a cloning-era workhorse. It contains coding material, but many workflows care more about its role as a reporter module than as a standalone protein-coding fragment. This is a genomic fragment, not a clean coding-sequence input. If you translate it naively, you hit stop codons because intron/exon context and strand assumptions matter.

BLAST-like search is one of the fastest ways to turn a mystery sequence into a shortlist of plausible stories.

Step-by-Step in Genome Forge

1. Use the included prebuilt bundle `docs/tutorial/datasets/case_bundles/case_aj/records.fasta` or regenerate it with `python3 docs/tutorial/datasets/extract_case_bundle.py --case AJ --out ./tmp/genomeforge_case_aj`.
2. Open the `Advanced` tab in Genome Forge and load: `EGFP_CDS, mCherry_CDS, lacZ_alpha_fragment, BRAF_exon15_fragment`.
3. Run `Run local similarity search against a small real-world panel to identify the most likely source of an unknown sequence.` once with default settings, then rerun after changing the query sequence or database panel.
4. Capture one screenshot of the main result panel so you can compare your run with the sample interpretation later.
5. Record the relevant endpoint(s): `/api/blast-search` and write one sentence explaining the biological takeaway.

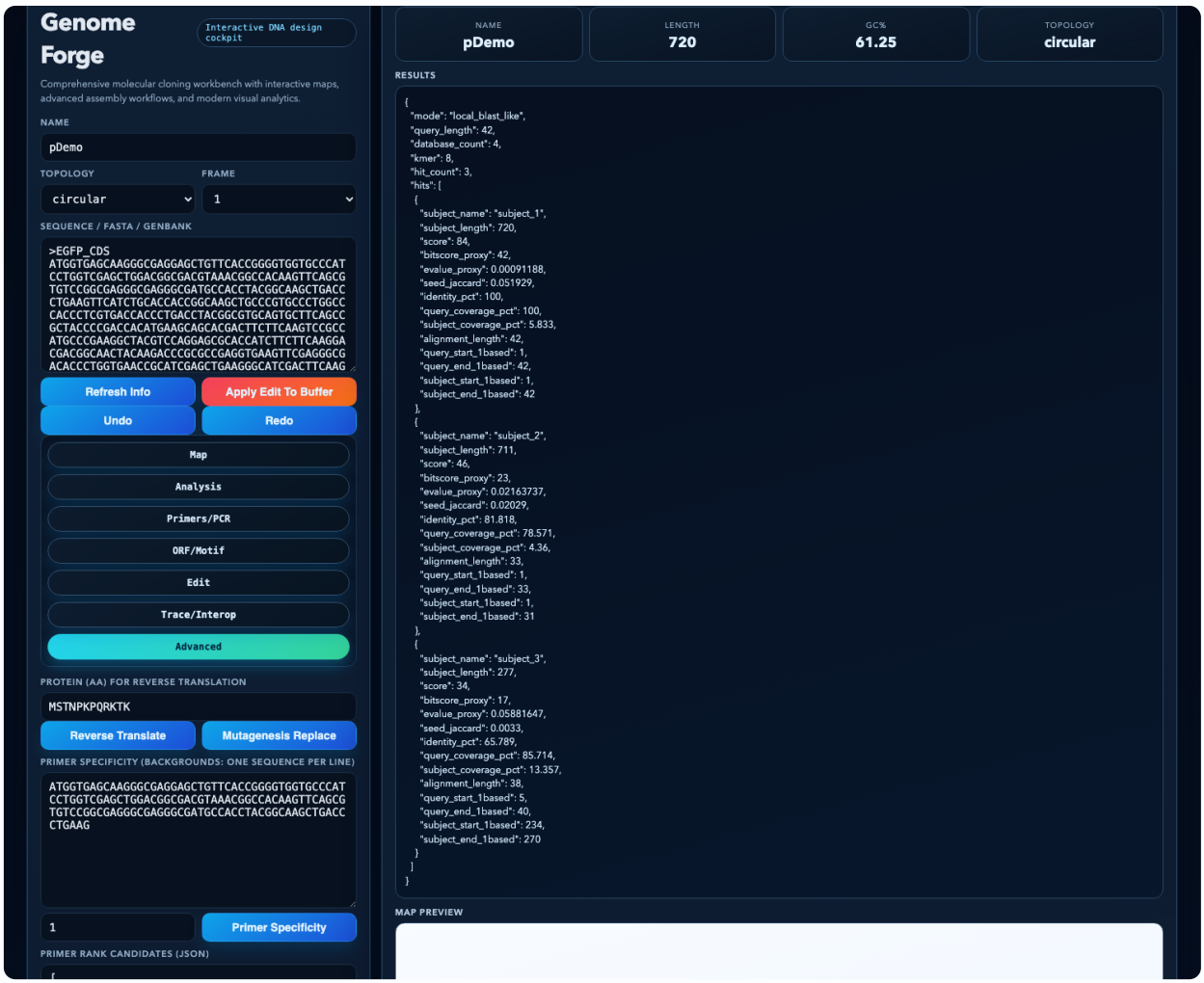


Figure 21. **BLAST-like identity search workflow**. Real UI screenshot from the local similarity-search case using the tutorial panel of EGFP, mCherry, lacZ, and BRAF. This is the kind of view you use to ask where an unknown sequence most plausibly came from.

Sample Results

Representative output shaped around the bundled real-world record(s) or their documented training derivatives. Values are rounded for readability, but the biological story is tied to the included data.


```
{
  "top_hit": "EGFP_CDS",
  "identity_pct": 100.0,
  "query_coverage_pct": 100.0,
  "runner_up": "mCherry_CDS"
}
```


Expected Results

- A ranked hit list with identity and coverage, not just a single best match.
- A narrative about what the top hit implies for sample identity or origin.
- A note explaining whether the hit pattern suggests clean identity or mixed origin.

How to Interpret the Results

- Full-length high-identity hits support strong identity claims.
- Partial hits are clues, not final answers; always inspect coverage.
- The runner-up hits often help explain contamination or domain sharing.

Biological Explanation

Similarity search is not just about identity percentages. Coverage, ranking, and context all matter. A high-identity partial hit can mean something very different from a full-length match, especially when you are trying to infer sample origin or contamination.

Fun fact from this example:

BLAST-like search is one of the fastest ways to turn a mystery sequence into a shortlist of plausible stories.

CASE AK · CLUSTER G

Case AK: Reference Element Auto-Flagging and siRNA Design/Mapping

How do reusable sequence libraries turn repeated manual annotation into a faster and more consistent design workflow?

TAB

Advanced

WORKFLOW

Reuse saved element libraries to auto-flag familiar sequence elements, then design and map siRNA candidates.

RECORDS

`EGFP_CDS`

`mCherry_CDS`

APIS

`/api/reference-db-save` `/api/reference-scan` `/api/sirna-design` `/api/sirna-map`

Why This Case Matters

Reporter CDS records are excellent for this because many labs annotate the same elements repeatedly. Saving reference libraries means the machine can recognize them quickly, and the same sequence can then be repurposed for knockdown-style thinking via siRNA design.

This case is about reuse. Bioinformatics becomes dramatically more efficient when previously understood sequence elements are captured as searchable reference knowledge rather than rediscovered each time.

Input Data Explained

This is a protein-coding DNA sequence (CDS). The sequence is meant to be translated from the first base, so codon boundaries and reading frame matter immediately. This is also a CDS, but it encodes a coral-derived red fluorescent protein rather than a GFP-family green reporter. That makes it useful for pairwise comparison and identity search.

The value of a reference library is partly speed, but mostly consistency: the same sequence gets recognized the same way every time.

Step-by-Step in Genome Forge

1. Use the included prebuilt bundle `docs/tutorial/datasets/case_bundles/case_ak/records.fasta` or regenerate it with `python3 docs/tutorial/datasets/extract_case_bundle.py --case AK --out ./tmp/genomeforge_case_ak`.
2. Open the `Advanced` tab in Genome Forge and load: `EGFP_CDS, mCherry_CDS`.
3. Run `Reuse saved element libraries to auto-flag familiar sequence elements, then design and map siRNA candidates.` once with default settings, then rerun after changing the reference library or siRNA ranking cutoff.
4. Capture one screenshot of the main result panel so you can compare your run with the sample interpretation later.
5. Record the relevant endpoint(s): `/api/reference-db-save, /api/reference-scan, /api/sirna-design, /api/sirna-map` and write one sentence explaining the biological takeaway.

Sample Results

Representative output shaped around the bundled real-world record(s) or their documented training derivatives. Values are rounded for readability, but the biological story is tied to the included data.


```
{
  "reference_hits": [
    "EGFP CDS",
    "mCherry CDS"
  ],
  "top_sirna_candidate_count": 5,
  "mapped_binding_sites": 5
}
```


Expected Results

- A reference-scan result showing which familiar elements were auto-flagged.
- A ranked siRNA candidate list with mapped target positions.
- A note explaining why reuse of reference knowledge reduces human error.

How to Interpret the Results

- Auto-flagging is strongest when the reference database is curated and versioned.
- siRNA ranking is still a prioritization tool; experimental validation remains necessary.
- The useful lesson is workflow reuse: annotation and design can feed each other.

Biological Explanation

This case is about reuse. Bioinformatics becomes dramatically more efficient when previously understood sequence elements are captured as searchable reference knowledge rather than rediscovered each time.

Fun fact from this example:

The value of a reference library is partly speed, but mostly consistency: the same sequence gets recognized the same way every time.

CASE AM · CLUSTER G

Case AM: Ambiguity-Aware Identity Search and Motif Rescue

If a sequence contains unresolved positions, can you still recover its likely identity and use it responsibly instead of discarding it as “bad data”?

TAB

Advanced

WORKFLOW

Treat an ambiguity-bearing consensus record as a real query and verify that identity search and motif logic still recover the correct biological family.

RECORDS

- EGFP_CDS
- EGFP_ambiguity_consensus_training
- mCherry_CDS

APIS

`/api/motif` `/api/blast-search` `/api/search-entities`

Why This Case Matters

The key input here is not a perfect sequence but a partially uncertain one. The ambiguity-bearing EGFP consensus stands in for a realistic intermediate artifact: a query that is clearly close to a known reporter family, yet still carries unresolved positions from sequencing or consensus assembly.

A huge amount of practical bioinformatics is about deciding what to do before the data are perfectly clean. Ambiguity codes let you represent uncertainty honestly. Ambiguity-aware search then lets you ask whether the uncertain record is still informative enough to identify, classify, or troubleshoot. The lesson is not that ambiguity disappears. The lesson is that uncertainty can still be computationally useful when represented explicitly.

Input Data Explained

This is a protein-coding DNA sequence (CDS). The sequence is meant to be translated from the first base, so codon boundaries and reading frame matter immediately. This record is still an EGFP-like coding sequence, but a few positions are encoded as ambiguity symbols such as R, Y, and N. That means the sequence stands for a small set of plausible molecules rather than one exact DNA string. This is also a CDS, but it encodes a coral-derived red fluorescent protein rather than a GFP-family green reporter. That makes it useful for pairwise comparison and identity search.

The scientific upgrade is subtle but important: you move from “this sequence is messy” to “this sequence still rules out many stories and supports a smaller plausible set.”

Starter Values

- Query record: `EGFP_ambiguity_consensus_training`
- Example motif query: `ATGGTGRG`
- Comparison panel: `EGFP_CDS, EGFP_ambiguity_consensus_training, mCherry_CDS`

Step-by-Step in Genome Forge

1. Use the included prebuilt bundle `docs/tutorial/datasets/case_bundles/case_am/records.fasta` or regenerate it with `python3 docs/tutorial/datasets/extract_case_bundle.py --case AM --out ./tmp/genomeforge_case_am`.
2. Open the `Advanced` tab in Genome Forge and load: `EGFP_CDS, EGFP_ambiguity_consensus_training, mCherry_CDS`.
3. Run `Treat an ambiguity-bearing consensus record as a real query and verify that identity search and motif logic still recover the correct biological family.` once with default settings, then rerun after changing the ambiguous query window or comparison panel.
4. Capture one screenshot of the main result panel so you can compare your run with the sample interpretation later.
5. Record the relevant endpoint(s): `/api/motif, /api/blast-search, /api/search-entities` and write one sentence explaining the biological takeaway.

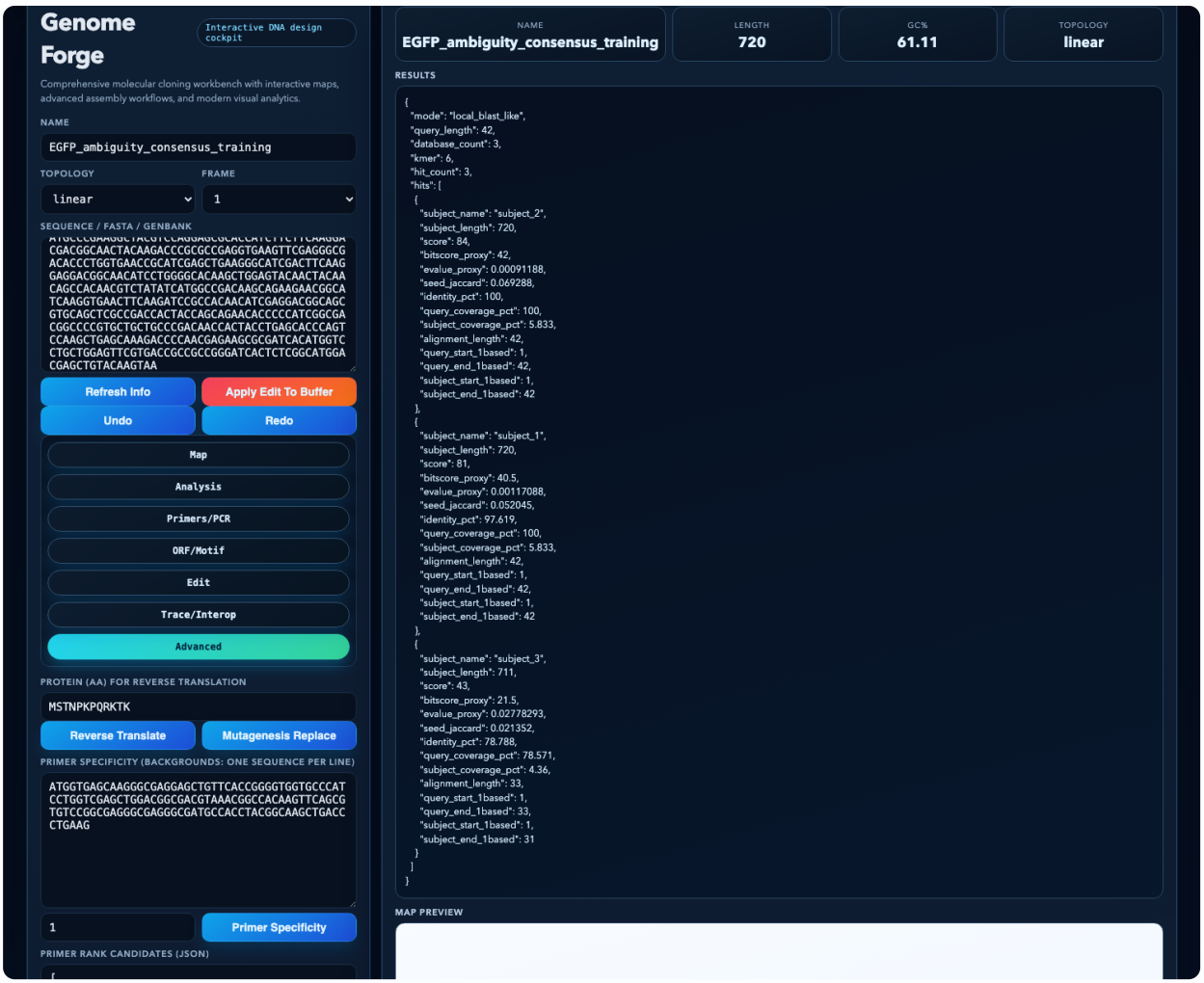


Figure 22. Ambiguity-aware identity search workflow. Real UI screenshot from the ambiguity-aware search lesson. The query itself carries unresolved positions, yet the search still recovers the correct reporter-family identity instead of treating the sequence as unusable.

Sample Results

Representative output shaped around the bundled real-world record(s) or their documented training derivatives. Values are rounded for readability, but the biological story is tied to the included data.


```
{
  "query_record": "EGFP_ambiguity_consensus_training",
  "motif_query": "ATGGTGRG",
  "top_blast_hit": "EGFP_CDS",
  "identity_pct": 100.0,
  "query_coverage_pct": 100.0,
  "runner_up": "mCherry_CDS",
  "interpretation": "uncertain positions did not erase reporter-family identity"
}
```


Expected Results

- A motif or similarity search in which an ambiguity-containing query still returns a biologically sensible top match.
- A ranked hit list that shows why the correct family remains the strongest explanation.
- A short statement separating what is still known confidently from what remains unresolved.

How to Interpret the Results

- Ambiguity-aware search should narrow the plausible identity space even when it cannot force every base to one final call.
- A strong top hit with high coverage means the uncertain sequence is still informative, not that uncertainty vanished.
- The right interpretation sounds like “this is still EGFP-family-like, with unresolved positions at specific sites,” not “the data are now magically exact.”

Biological Explanation

A huge amount of practical bioinformatics is about deciding what to do before the data are perfectly clean. Ambiguity codes let you represent uncertainty honestly. Ambiguity-aware search then lets you ask whether the uncertain record is still informative enough to identify, classify, or troubleshoot. The lesson is not that ambiguity disappears. The lesson is that uncertainty can still be computationally useful when represented explicitly.

Fun fact from this example:

The scientific upgrade is subtle but important: you move from “this sequence is messy” to “this sequence still rules out many stories and supports a smaller plausible set.”

CLUSTER H

Cluster H: Reproducibility, Governance, and Delivery

How to turn one-off sequence work into something another scientist can review, rerun, and trust.

Lessons in This Cluster

- Case L · Collaboration, Audit, and Review Governance
- Case T · Batch Reproducibility and Parameter Locking
- Case AB · Reproducible Report Package
- Case AC · Parameter Sensitivity and Robustness Check
- Case AD · End-to-End Release Checklist and Handoff

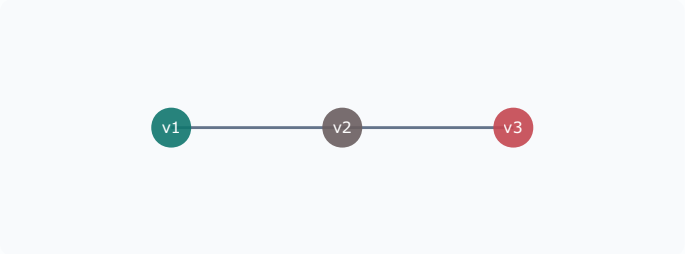


Figure 23. Good sequence work is a product, not just a result: it needs history, review, packaging, and handoff.

CASE L · CLUSTER H

Case L: Collaboration, Audit, and Review Governance

How do you make sequence work reviewable by another person instead of leaving it as personal screen state?

TAB

Advanced

WORKFLOW

Create a workspace, assign roles, and run a simple review flow on a saved construct project.

RECORDS

`EGFP_CDS`

APIS

`/api/workspace-create` `/api/project-permissions` `/api/review-submit` `/api/review-approve`

Why This Case Matters

This governance cluster deliberately uses a familiar record so the tutorial attention stays on process rather than molecular interpretation. The point is to show how sequence work becomes team knowledge.

Scientific reproducibility depends on more than file storage. Roles, review, audit trails, and explicit approval states are part of the computational record. Without them, a project may be technically complete but socially fragile.

Input Data Explained

This is a protein-coding DNA sequence (CDS). The sequence is meant to be translated from the first base, so codon boundaries and reading frame matter immediately.

In modern labs, the "truth" of a construct often lives partly in people and partly in software; governance features are how you keep those from drifting apart.

Step-by-Step in Genome Forge

1. Use the included prebuilt bundle `docs/tutorial/datasets/case_bundles/case_l/records.fasta` or regenerate it with `python3 docs/tutorial/datasets/extract_case_bundle.py --case L --out ./tmp/genomeforge_case_l`.
2. Open the `Advanced` tab in Genome Forge and load: `EGFP_CDS`.
3. Run `Create a workspace, assign roles, and run a simple review flow on a saved construct project.` once with default settings, then rerun after changing the assigned role or review state.
4. Capture one screenshot of the main result panel so you can compare your run with the sample interpretation later.
5. Record the relevant endpoint(s): `/api/workspace-create, /api/project-permissions, /api/review-submit, /api/review-approve` and write one sentence explaining the biological takeaway.

Sample Results

Representative output shaped around the bundled real-world record(s) or their documented training derivatives. Values are rounded for readability, but the biological story is tied to the included data.


```
{
  "workspace_created": true,
  "roles": {
    "owner_user": "owner",
    "editor_user": "editor",
    "reviewer_user": "reviewer"
  },
  "review_status": "approved"
}
```


Expected Results

- A saved project with explicit role assignments and a traceable review event.
- An audit-friendly record of who changed or approved what.
- A clear explanation of why governance matters for scientific trust.

How to Interpret the Results

- A sequence project that cannot be reviewed cleanly is harder to trust and harder to reuse.
- Audit logs are most valuable when something becomes confusing later; build them before confusion arrives.
- Governance features are scientific infrastructure, not bureaucracy for its own sake.

Biological Explanation

Scientific reproducibility depends on more than file storage. Roles, review, audit trails, and explicit approval states are part of the computational record. Without them, a project may be technically complete but socially fragile.

Fun fact from this example:

In modern labs, the “truth” of a construct often lives partly in people and partly in software; governance features are how you keep those from drifting apart.

CASE T · CLUSTER H

Case T: Batch Reproducibility and Parameter Locking

How do you make sure that differences between records reflect biology instead of accidental parameter drift?

TAB

Advanced

WORKFLOW

Run the same logic across several records with a locked parameter set so outputs stay comparable.

RECORDS

- EGFP_CDS
- mCherry_CDS
- BRAF_exon15_fragment

APIS

`/api/project-save` `/api/sequence-analytics` `/api/batch-digest`

Why This Case Matters

Batch work is where reproducibility discipline becomes visible. Using several real records with one locked configuration lets you compare outputs honestly instead of wondering whether the settings changed between runs.

Parameter locking matters because software is part of the experiment. If settings drift invisibly, your comparison is no longer about molecules alone. Reproducibility begins when you can state exactly what was held constant.

Input Data Explained

This is a protein-coding DNA sequence (CDS). The sequence is meant to be translated from the first base, so codon boundaries and reading frame matter immediately. This is also a CDS, but it encodes a coral-derived red fluorescent protein rather than a GFP-family green reporter. That makes it useful for pairwise comparison and identity search. This is a genomic fragment, not a clean coding-sequence input. If you translate it naively, you hit stop codons because intron/exon context and strand assumptions matter.

A batch run is only comparable if the software treated each input under the same contract.

Step-by-Step in Genome Forge

1. Use the included prebuilt bundle `docs/tutorial/datasets/case_bundles/case_t/records.fasta` or regenerate it with `python3 docs/tutorial/datasets/extract_case_bundle.py --case T --out ./tmp/genomeforge_case_t`.
2. Open the `Advanced` tab in Genome Forge and load: `EGFP_CDS, mCherry_CDS, BRAF_exon15_fragment`.
3. Run `Run the same logic across several records with a locked parameter set so outputs stay comparable.` once with default settings, then rerun after changing one parameter intentionally to test robustness.
4. Capture one screenshot of the main result panel so you can compare your run with the sample interpretation later.
5. Record the relevant endpoint(s): `/api/project-save, /api/sequence-analytics, /api/batch-digest` and write one sentence explaining the biological takeaway.

Sample Results

Representative output shaped around the bundled real-world record(s) or their documented training derivatives. Values are rounded for readability, but the biological story is tied to the included data.


```
{
  "record_count": 3,
  "parameter_profile": "locked",
  "comparison_ready": true
}
```


Expected Results

- A batch run with identical settings applied to multiple records.
- A written record of the locked parameter profile.
- A statement about what differences can now be attributed to biology rather than settings.

How to Interpret the Results

- Locked parameters create fair comparisons.
- If a rerun needs different settings, treat it as a new experiment and say so.
- Reproducibility is easiest when the configuration is explicit and boring.

Biological Explanation

Parameter locking matters because software is part of the experiment. If settings drift invisibly, your comparison is no longer about molecules alone. Reproducibility begins when you can state exactly what was held constant.

Fun fact from this example:

A batch run is only comparable if the software treated each input under the same contract.

CASE AB · CLUSTER H

Case AB: Reproducible Report Package

What does a handoff artifact look like when you want another person to inspect the same biological object, not just hear about it?

TAB

Advanced

WORKFLOW

Package a saved project and a share bundle so another scientist can reopen the same analysis context.

RECORDS

`EGFP_CDS`

`pUC19_MCS`

APIS

`/api/project-save` `/api/share-create` `/api/share-load`

Why This Case Matters

This case treats the molecular record as a deliverable. Using a familiar reporter/vector pair keeps the content concrete while shifting the lesson toward packaging and transport of scientific context.

A reproducible report package includes the molecule, the interpretation, and the route back to the evidence. Sharing only screenshots or only FASTA is usually not enough. The useful package is the one another person can actually reopen and interrogate.

Input Data Explained

This is a protein-coding DNA sequence (CDS). The sequence is meant to be translated from the first base, so codon boundaries and reading frame matter immediately. This sequence is short, circularly interpreted, and packed with restriction motifs. It is intentionally synthetic and engineered for manipulation rather than natural gene expression.

A good handoff is a kindness to your future self as much as to your collaborators.

Step-by-Step in Genome Forge

1. Use the included prebuilt bundle `docs/tutorial/datasets/case_bundles/case_ab/records.fasta` or regenerate it with `python3 docs/tutorial/datasets/extract_case_bundle.py --case AB --out ./tmp/genomeforge_case_ab`.
2. Open the `Advanced` tab in Genome Forge and load: `EGFP_CDS, pUC19_MCS`.
3. Run `Package a saved project and a share bundle so another scientist can reopen the same analysis context.` once with default settings, then rerun after changing whether content is embedded in the bundle.
4. Capture one screenshot of the main result panel so you can compare your run with the sample interpretation later.
5. Record the relevant endpoint(s): `/api/project-save, /api/share-create, /api/share-load` and write one sentence explaining the biological takeaway.

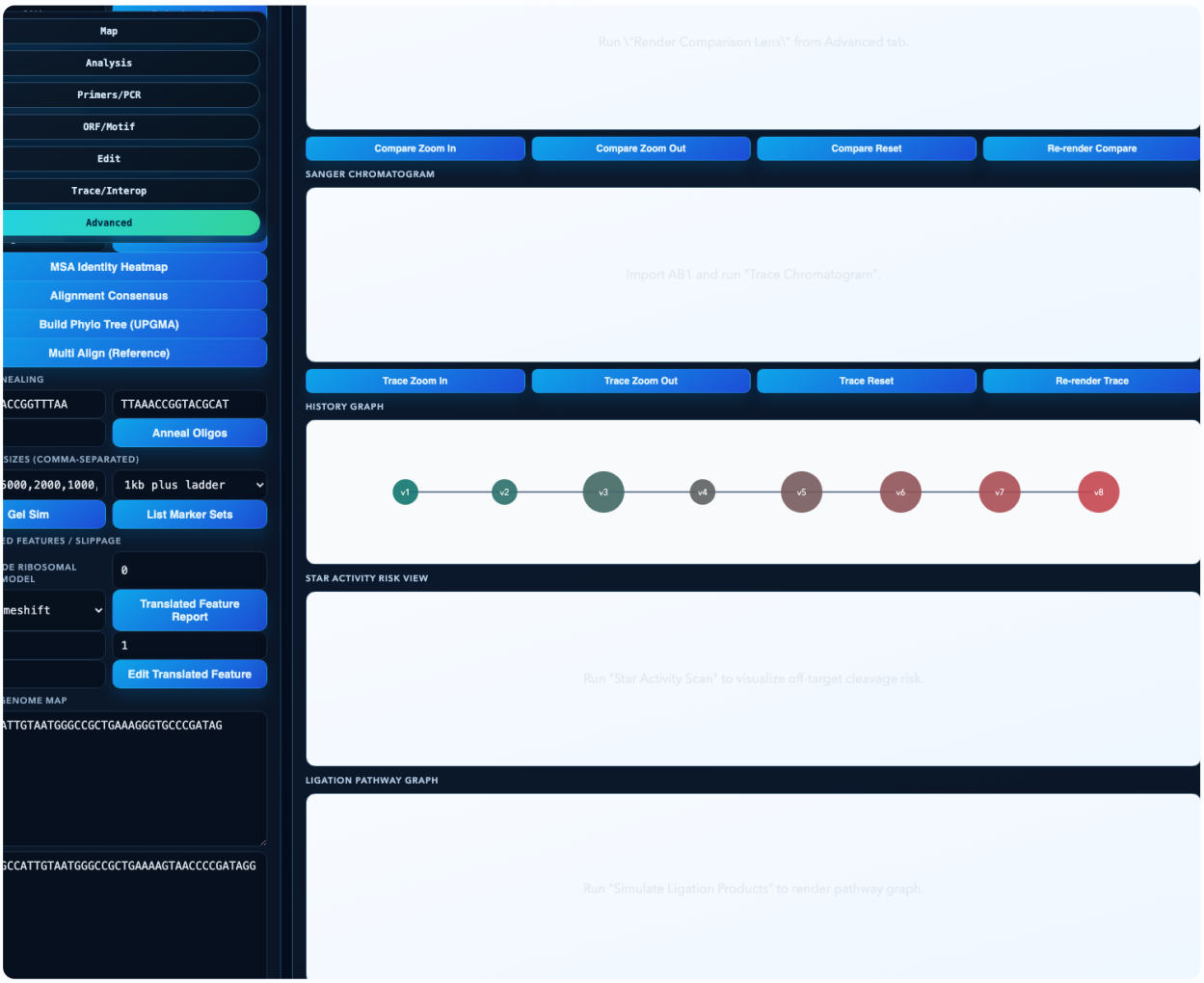


Figure 24. **Project-history and reproducibility workflow**. Real UI screenshot from a saved-project history case. This view matters because sequence work becomes more trustworthy when design state and revision history stay attached.

Sample Results

Representative output shaped around the bundled real-world record(s) or their documented training derivatives. Values are rounded for readability, but the biological story is tied to the included data.


```
{
  "project_saved": true,
  "share_bundle_created": true,
  "project_count": 1,
  "handoff_ready": true
}
```


Expected Results

- A saved project plus a reloadable share bundle.
- A note describing what context the package preserves.
- A simple check that another user or browser session can reopen the artifact.

How to Interpret the Results

- A package is only reproducible if it can be reopened independently of your current browser state.
- Think of share bundles as portable scientific state, not just exported files.
- The less hidden context a handoff requires, the better the handoff.

Biological Explanation

A reproducible report package includes the molecule, the interpretation, and the route back to the evidence. Sharing only screenshots or only FASTA is usually not enough. The useful package is the one another person can actually reopen and interrogate.

Fun fact from this example:

A good handoff is a kindness to your future self as much as to your collaborators.

CASE AC · CLUSTER H

Case AC: Parameter Sensitivity and Robustness Check

Would you reach the same biological conclusion if a reasonable analyst chose slightly different parameters?

TAB

Advanced

WORKFLOW

Rerun a workflow under a small parameter sweep to see whether the biological conclusion is robust or fragile.

RECORDS

`BRAF_exon15_fragment`

`EGFP_CDS`

APIS

`/api/primer-design` `/api/grna-design` `/api/sequence-analytics`

Why This Case Matters

This case is built around the uncomfortable but essential idea that a strong pipeline should tolerate small setting changes. Real biological examples help here because the outputs matter more when the records are not invented toys.

Sensitivity analysis is how you keep yourself honest. If a conclusion flips under small parameter nudges, the conclusion is fragile and should be reported as such. Robustness is not a brag; it is a property you test.

Input Data Explained

This is a genomic fragment, not a clean coding-sequence input. If you translate it naively, you hit stop codons because intron/exon context and strand assumptions matter. This is a protein-coding DNA sequence (CDS). The sequence is meant to be translated from the first base, so codon boundaries and reading frame matter immediately.

Some of the best scientific writing consists of one calm sentence saying, “this result is parameter-sensitive, so treat it cautiously.”

Step-by-Step in Genome Forge

1. Use the included prebuilt bundle `docs/tutorial/datasets/case_bundles/case_ac/records.fasta` or regenerate it with `python3 docs/tutorial/datasets/extract_case_bundle.py --case AC --out ./tmp/genomeforge_case_ac`.
2. Open the `Advanced` tab in Genome Forge and load: `BRAF_exon15_fragment, EGFP_CDS`.
3. Run `Rerun a workflow under a small parameter sweep to see whether the biological conclusion is robust or fragile.` once with default settings, then rerun after changing one threshold across a small sweep.
4. Capture one screenshot of the main result panel so you can compare your run with the sample interpretation later.
5. Record the relevant endpoint(s): `/api/primer-design, /api/grna-design, /api/sequence-analytics` and write one sentence explaining the biological takeaway.

Sample Results

Representative output shaped around the bundled real-world record(s) or their documented training derivatives. Values are rounded for readability, but the biological story is tied to the included data.


```
{
  "parameter_sweep_size": 3,
  "robust_call_examples": [
    "EGFP length and ORF remain stable"
  ],
  "fragile_call_example": "borderline primer ranking can flip"
}
```


Expected Results

- A mini-sweep with at least one stable output and one potentially fragile output.
- A note about which conclusions remain trustworthy across settings.
- A recommendation for how to report or mitigate fragile results.

How to Interpret the Results

- Stable outputs earn confidence; fragile outputs earn caution.
- A parameter-sensitive result is not useless, but it should be presented with narrower claims.
- Sensitivity analysis is part of interpretation, not just a software exercise.

Biological Explanation

Sensitivity analysis is how you keep yourself honest. If a conclusion flips under small parameter nudges, the conclusion is fragile and should be reported as such. Robustness is not a brag; it is a property you test.

Fun fact from this example:

Some of the best scientific writing consists of one calm sentence saying, “this result is parameter-sensitive, so treat it cautiously.”

CASE AD · CLUSTER H

Case AD: End-to-End Release Checklist and Handoff

If you had to stop work today and let another person continue tomorrow, what would they need?

TAB

Advanced

WORKFLOW

Treat the tutorial workspace like a releasable scientific software product and verify the handoff boundary.

RECORDS

- EGFP_CDS
- mCherry_CDS
- pUC19_MCS
- BRAF_exon15_fragment

APIS

`/api/project-save` `/api/share-create` `/api/project-history-svg`

Why This Case Matters

The final case intentionally zooms out. The real-world molecules are now ingredients in a process story: save state, capture provenance, package outputs, and document what still needs verification. That is what mature scientific computing looks like.

End-to-end handoff is where software engineering and biology meet most directly. A good package includes executable steps, sample data, known limitations, and enough context that a new person can rerun the work without guessing what mattered.

Input Data Explained

This is a protein-coding DNA sequence (CDS). The sequence is meant to be translated from the first base, so codon boundaries and reading frame matter immediately. This is also a CDS, but it encodes a coral-derived red fluorescent protein rather than a GFP-family green reporter. That makes it useful for pairwise comparison and identity search. This sequence is short, circularly interpreted, and packed with restriction motifs. It is intentionally synthetic and engineered for manipulation rather than natural gene expression. This is a genomic fragment, not a clean coding-sequence input. If you translate it naively, you hit stop codons because intron/exon context and strand assumptions matter.

A handoff is successful when the next person says “I know where to start,” not when they say “I have the files.”

Step-by-Step in Genome Forge

1. Use the included prebuilt bundle `docs/tutorial/datasets/case_bundles/case_ad/records.fasta` or regenerate it with `python3 docs/tutorial/datasets/extract_case_bundle.py --case AD --out ./tmp/genomeforge_case_ad`.
2. Open the `Advanced` tab in Genome Forge and load: `EGFP_CDS, mCherry_CDS, pUC19_MCS, BRAF_exon15_fragment`.
3. Run `Treat the tutorial workspace like a releasable scientific software product and verify the handoff boundary.` once with default settings, then rerun after changing the handoff checklist or packaged artifacts.
4. Capture one screenshot of the main result panel so you can compare your run with the sample interpretation later.
5. Record the relevant endpoint(s): `/api/project-save, /api/share-create, /api/project-history-svg` and write one sentence explaining the biological takeaway.

Sample Results

Representative output shaped around the bundled real-world record(s) or their documented training derivatives. Values are rounded for readability, but the biological story is tied to the included data.


```
{
  "checklist_items": [
    "sample data bundled",
    "tutorial regenerated",
    "PDF built",
    "tests passed"
  ],
  "handoff_state": "ready"
}
```


Expected Results

- A release-style checklist that covers data, docs, state, and verification.
- A clear statement of what remains uncertain or intentionally simplified.
- A handoff artifact that helps the next person continue without hidden memory.

How to Interpret the Results

- The final deliverable is not the molecule alone; it is the reproducible workflow around the molecule.
- The best handoff documents both what works and what is still risky.
- Zero-memory pickup is a great standard for scientific software because people and projects always get interrupted.

Biological Explanation

End-to-end handoff is where software engineering and biology meet most directly. A good package includes executable steps, sample data, known limitations, and enough context that a new person can rerun the work without guessing what mattered.

Fun fact from this example:

A handoff is successful when the next person says “I know where to start,” not when they say “I have the files.”